JavaScript 入門

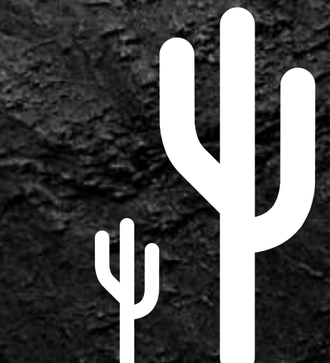
歡迎大家在這裡提問

課程 Slido

講師介紹

鴕鳥

- JavaScript 入門講師



大家有學過程式語言嗎？

JavaScript 是什麼?

JavaScript（JS）可以在網頁中加入互動，讓靜態的網頁的產生一些變化。

<script> 標籤

- JavaScript 程式碼會寫在在 HTML 的 <script> 中

```
<html>
    <head>

    </head>
    <body>
        <script>
            console.log("Hello world");
        </script>
    </body>
</html>
```

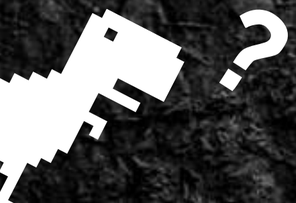
註解（Comments）

- 單行註解:

```
//console.log("Hello world");
```

- 多行註解:

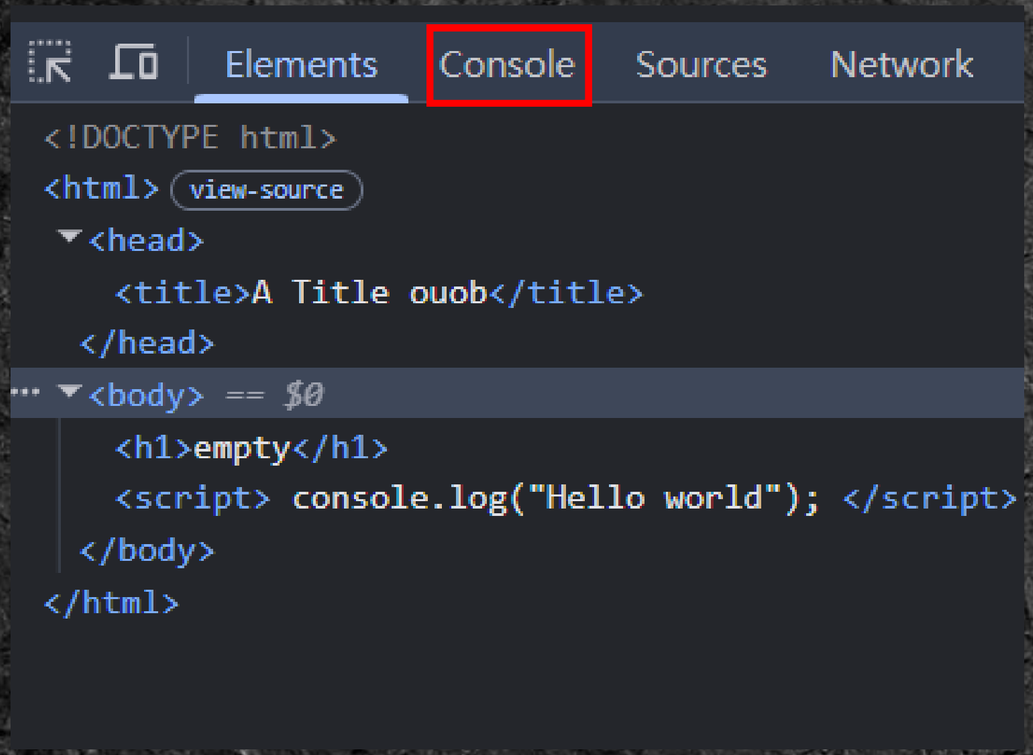
```
/*
for(let i = 0;i < 10;i+=1){
    console.log(i);
}
*/
```

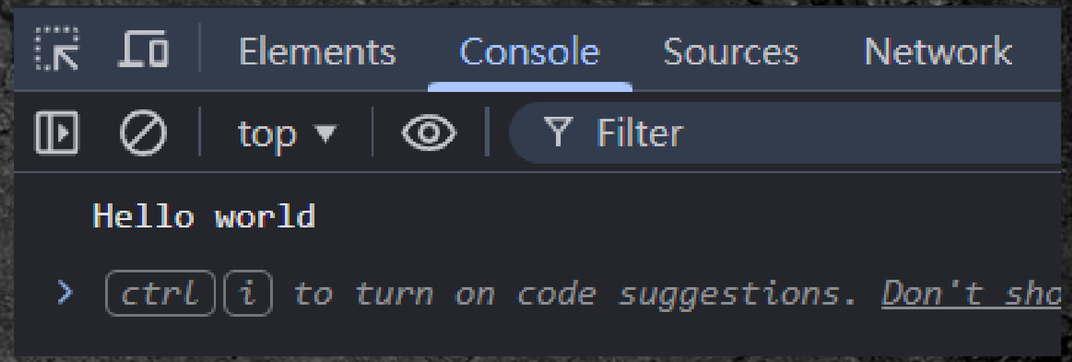


輸出 與 Console

Console （控制台）

- JavaScript 的執行狀況、 錯誤訊息都會記錄在 Console





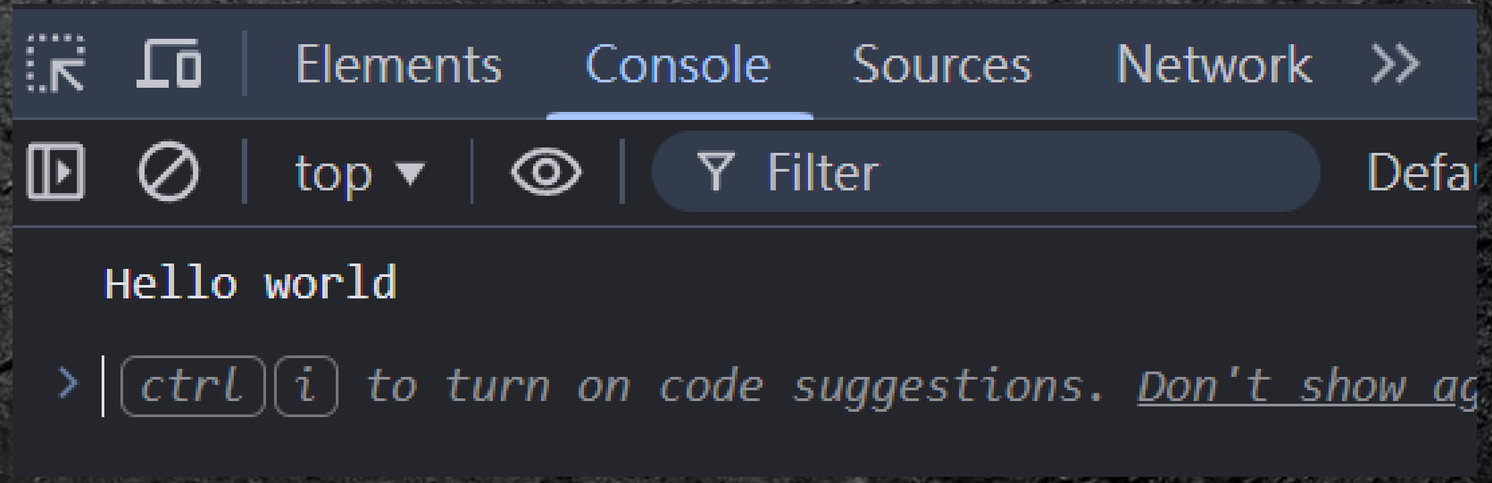
輸出： console.log();

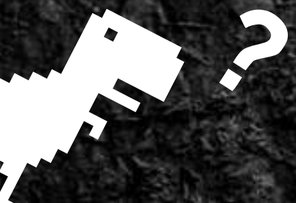
```
//在 console 上輸出「Hello world」
console.log("Hello world");
```

- "Hello world" → 一段文字（字串）

輸出： console.log();

```
//在 console 上輸出「Hello world」
console.log("Hello world");
```





Task 01 （3分鐘）

在 Console 上輸出 Hello world

```
//在 console 上輸出「Hello world」
console.log("Hello world");
```

把分號；刪掉會有什麼改變？

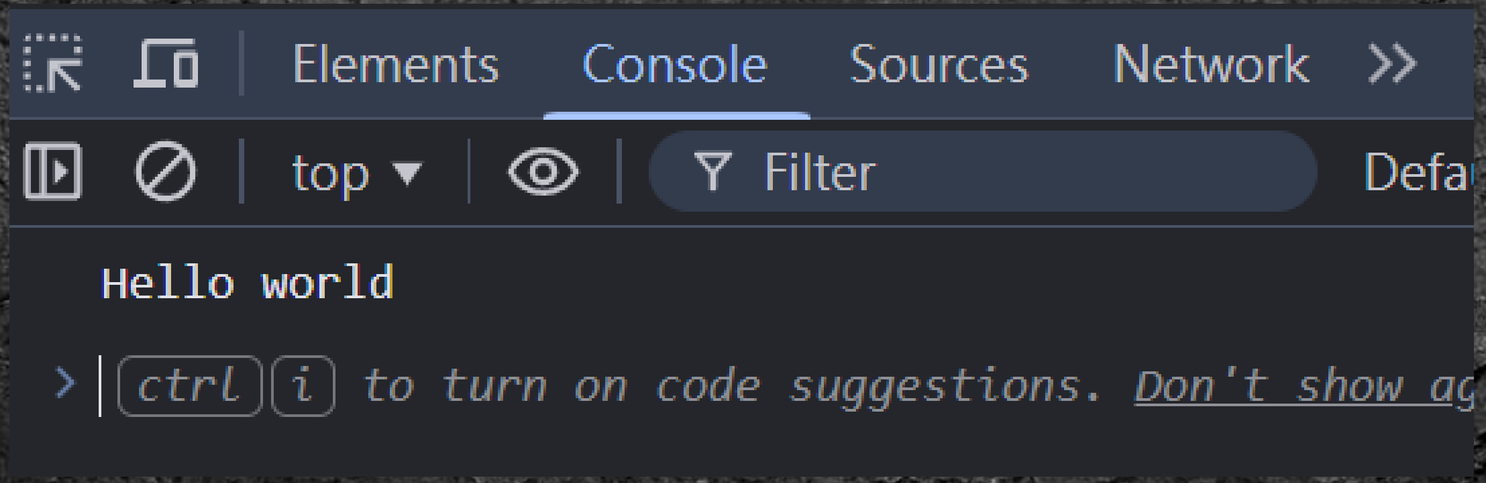
```
console.log("Hello world");
```

```
console.log("Hello world")
```

好耶，程式一樣能跑

```
console.log("Hello world");
```

```
console.log("Hello world")
```



所以我們可以不加分號嗎？

所以我們可以不加分號嗎？

為什麼要加分號；

```
//在 console 上輸出「Hello world」
console.log("Hello world");
```

- ；代表一段程式的結束
- Automatic Semicolon Insertion → JS 自動補分號的機制

為什麼要加分號；

```
//在 console 上輸出「Hello world」
console.log("Hello world");
```

一定要加上分號，避免跑出奇怪的錯誤!

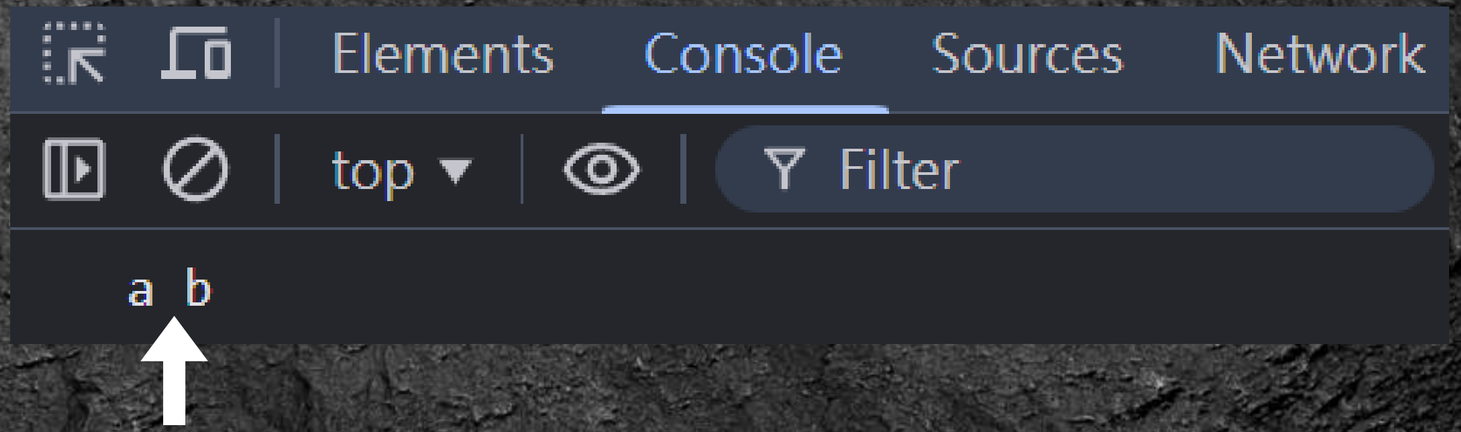
一次輸出多個東西

```
console.log("a", "b");
```

- 輸出兩組文字，"a" 和 "b"

一次輸出多個東西

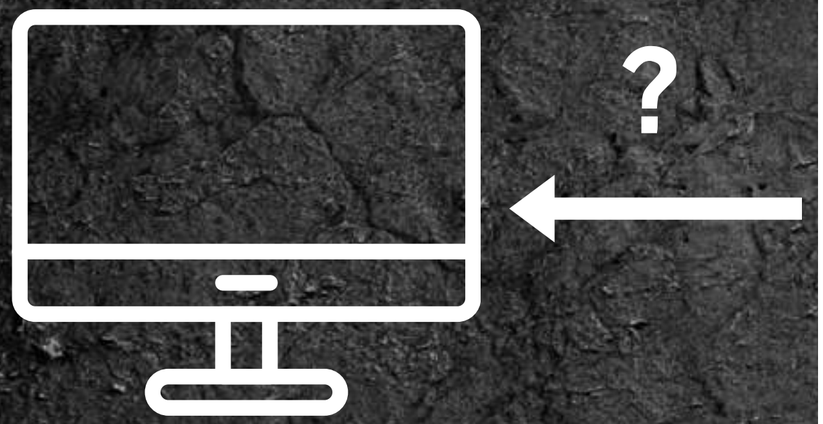
```
console.log("a", "b");
```



輸出時兩個資料之間會自動補空格

變數、指定運算與算數運算

網頁要如何記錄使用者的輸入?



?

台北、嘉義

1, 2

台北、嘉義

1, 2

放「文字」的變數

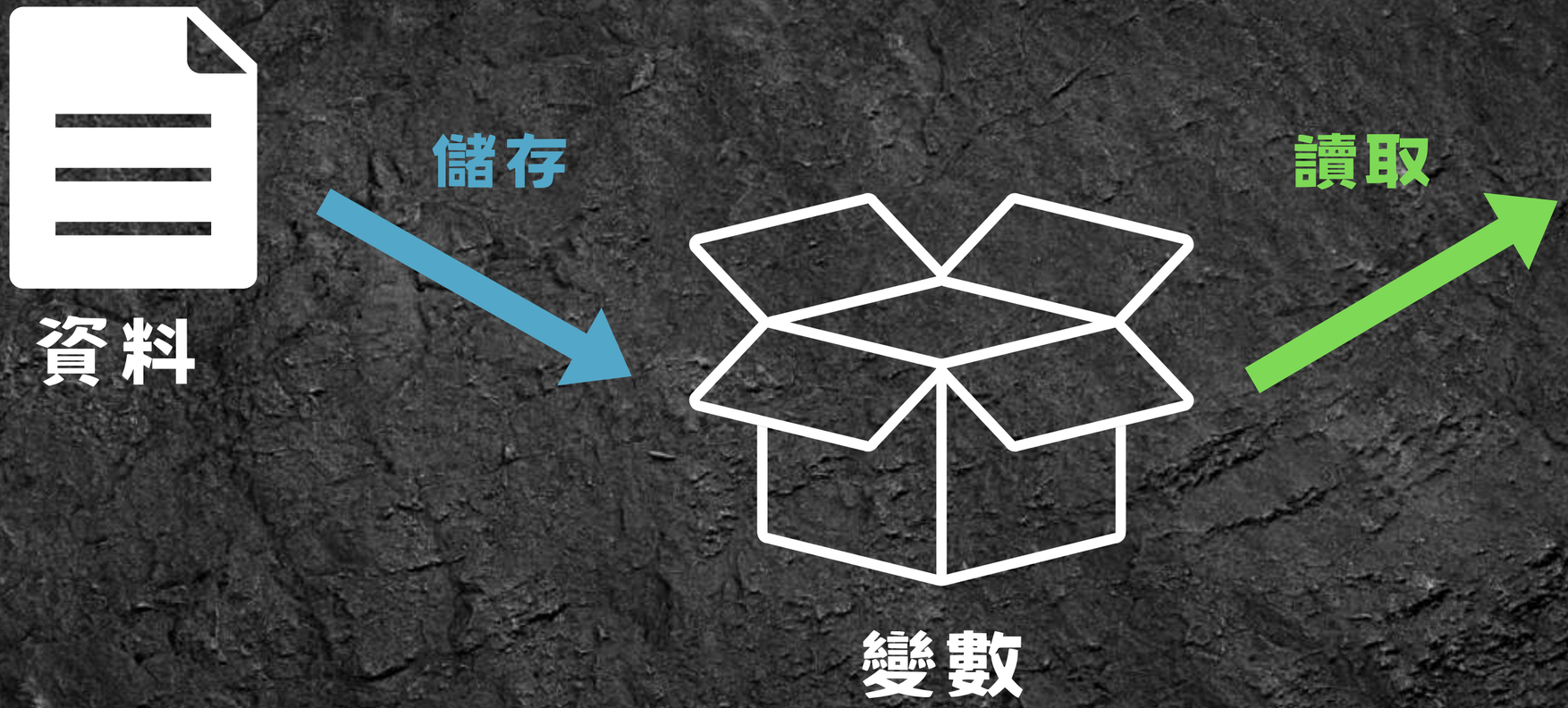
放「數字」的變數

台北、嘉義

1, 2

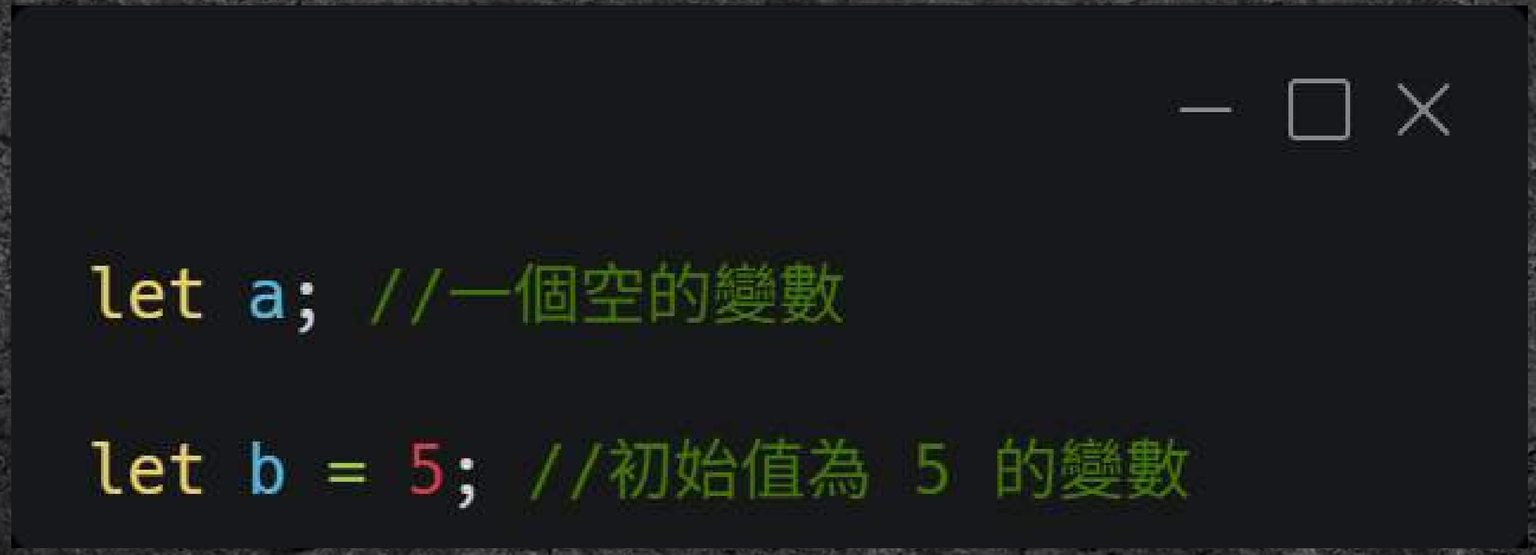
變數（Variables）

- 儲存資料的容器



宣告變數

- let
- 不要用「保留字」來命名變數（e.g. let，true）



```
let a; //一個空的變數

let b = 5; //初始值為 5 的變數
```

一些 JavaScript 保留字

abstract	arguments	boolean	break	byte
case	catch	char	class*	const
continue	debugger	default	delete	do
double	else	enum*	eval	export*
extends*	false	final	finally	float
for	function	goto	if	implements
import*	in	instanceof	int	interface
let	long	native	new	null
package	private	protected	public	return
short	static	super*	switch	synchronized
this	throw	throws	transient	true
try	typeof	var	void	volatile
while	with	yield		

指定運算、賦值（Assignment）

- =
- 將指定的東西丟給變數

```
//宣告 a, b 兩個變數
let a = 2, b = 10;

a = 5; //把 a 的值改為 5

a = b; //把 b 的值複製丟給 a

a = b + 3; //把一個運式的結果丟給 a
```

變數的資料型態

字串 （String）	"Hello world", 'ouob'
數字 （Number）	1, 2147483647
布林值 （Boolean）	true(對), false(錯)

字串（String）

- 儲存文字
- 文字加上「""」或「''」代表是字串

e.g. "Hello world" , 'Yes! BanG_Dream!'

字串（String）

- 字串可以用「+」連接

e.g. "Fire " + "Bird" = "Fire Bird"

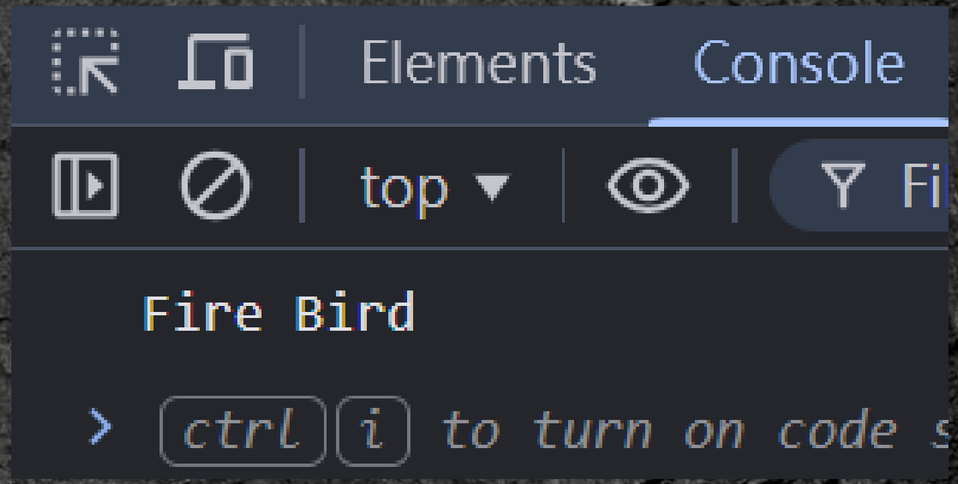
```
let text = "Fire " + "Bird";
console.log(text);
```

字串（String）

- 字串可以用「+」連接

e.g. "Fire " + "Bird" = "Fire Bird"

```
let text = "Fire " + "Bird";
console.log(text);
```



```
let text = "Fire " + "Bird";
console.log(text);
```

數字（Number）

- 儲存數字
- 可以做算術運算（加法、減法、…）

```
let Int = 10;
let Float = 3.14;
```

算術運算

加法 + 、減法 - 、乘法 * 、除法 /
次方 ** 、取餘數 %
括號 ()

- JS 的四則運算規則

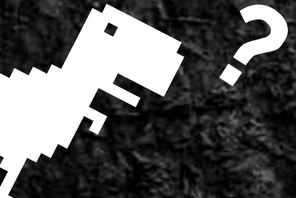
優先順序: () > ** > * , / , % > + , -

算術運算

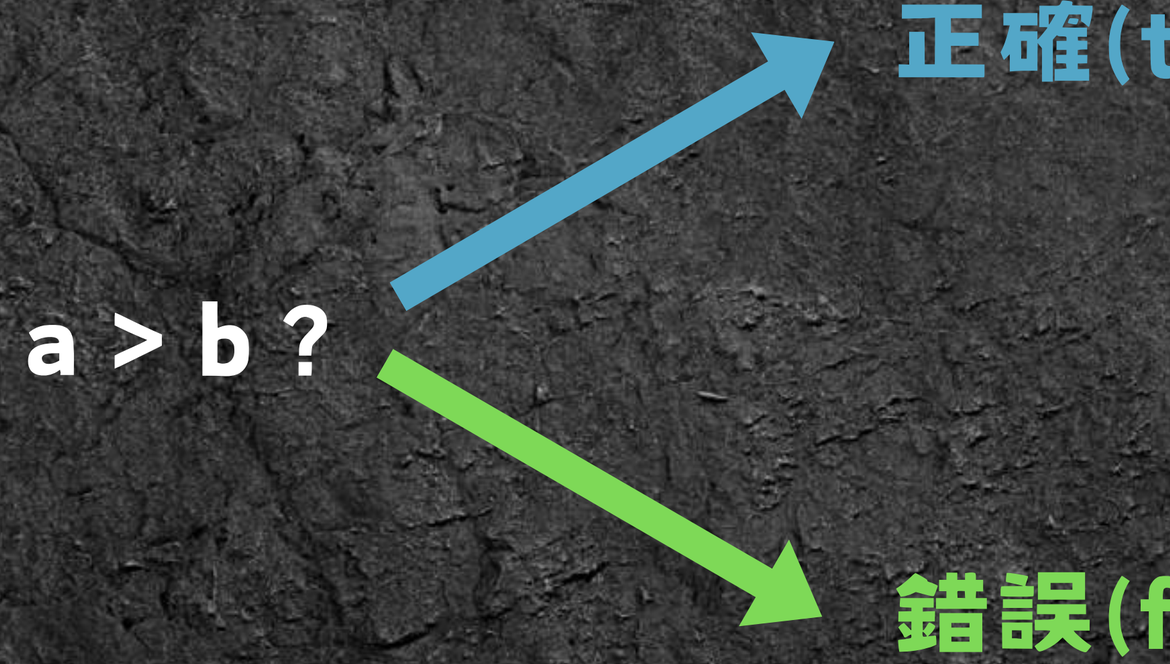
```
a = a + 5;
a = a - 5;
a = a * 5;
a = a / 5;
a = a % 5;
a = a ** 5;
```

簡化

```
a += 5;
a -= 5;
a *= 5;
a /= 5;
a %= 5;
a **= 5;
```

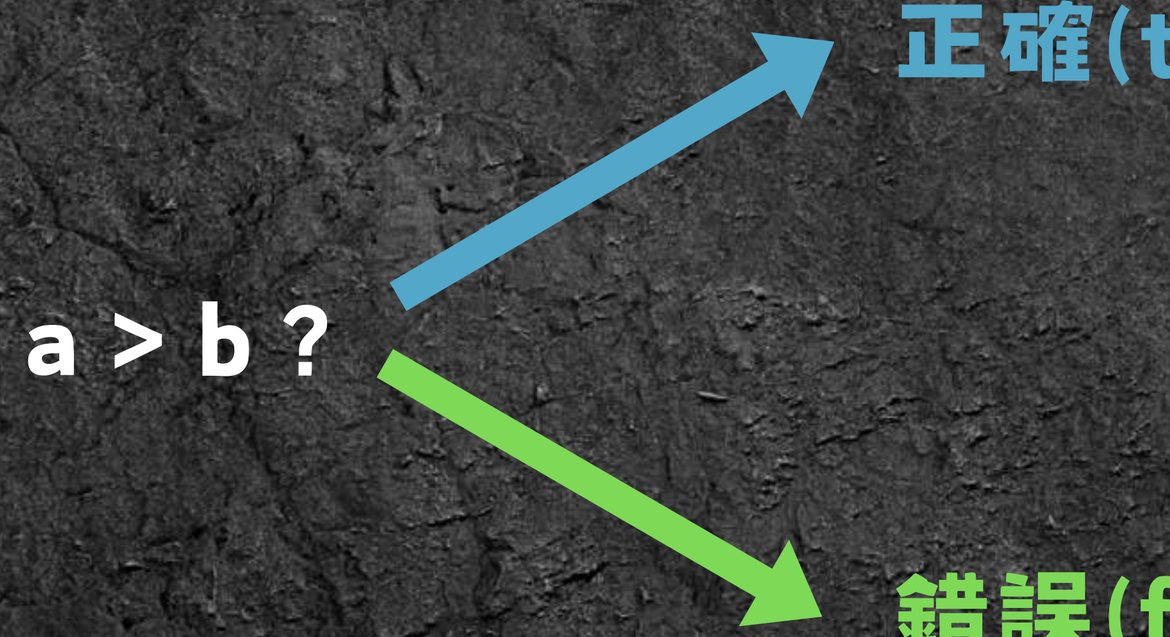


我們要怎麼記錄一件事的正確性?



我們要怎麼記錄一件事的正確性?

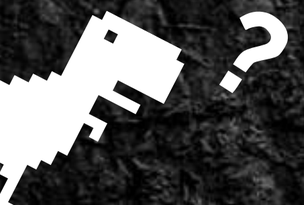
→ 布林值（Boolean）



布林值（Boolean）

- 只有 true，false 兩種值
- 要注意大小寫 ✅ true false ❌ True False

```
let a = true;

let b = false;
```

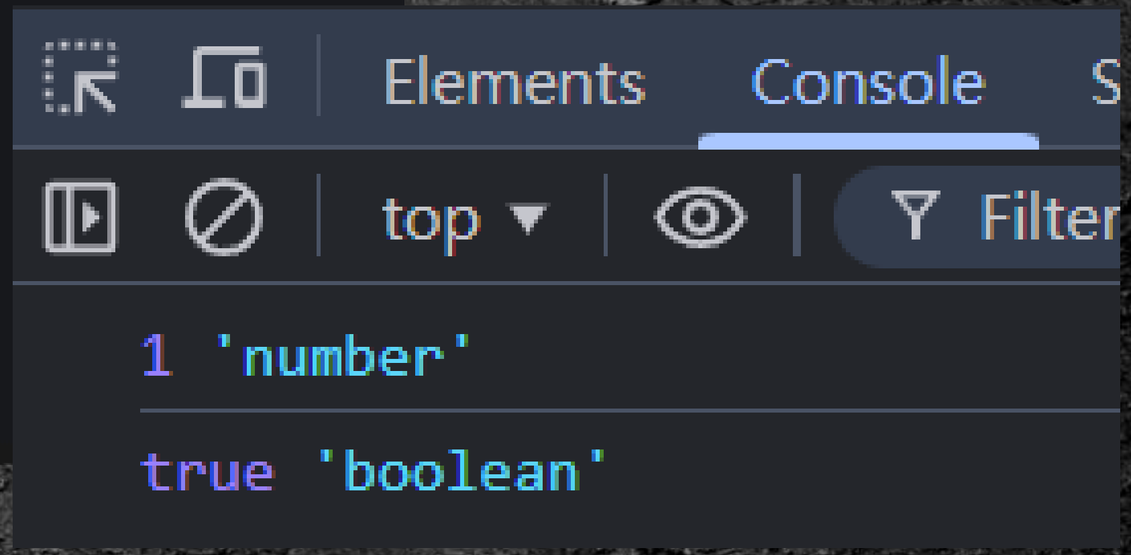


- typeof ：用來檢查變數的資料型態

```
let a = 1;
console.log(a, typeof a);
a = true;
console.log(a, typeof a);
```

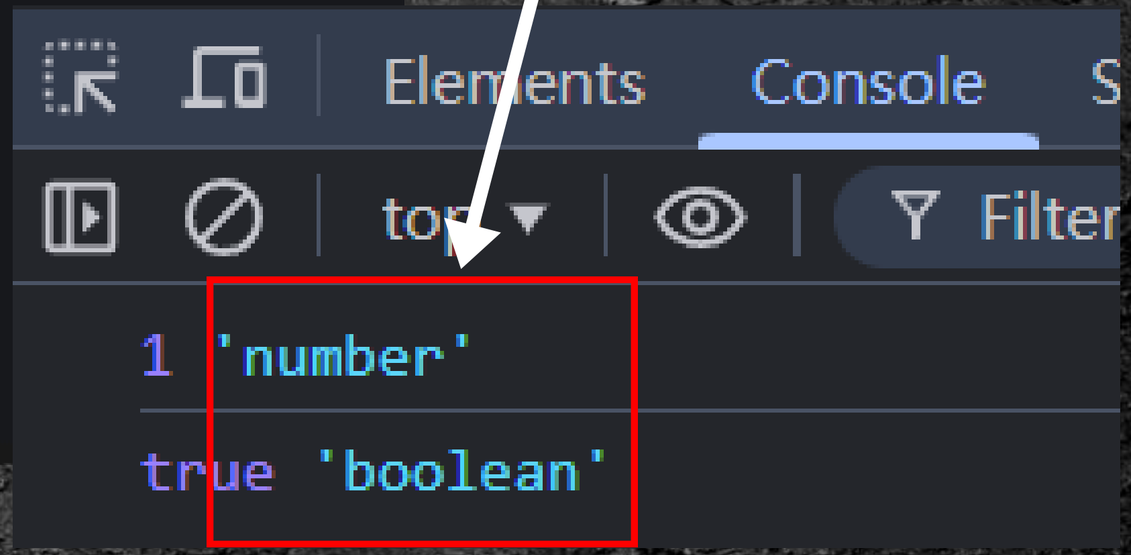
- typeof ：用來檢查變數的資料型態

```
let a = 1;
console.log(a, typeof a);
a = true;
console.log(a, typeof a);
```



- typeof ：用來檢查變數的資料型態
- 變數的資料型態會隨著儲存的資料而改變

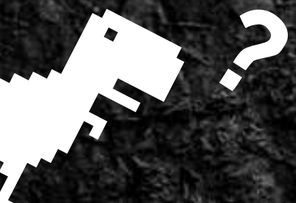
```
let a = 1;
console.log(a, typeof a);
a = true;
console.log(a, typeof a);
```



let vs. const （constant常數）

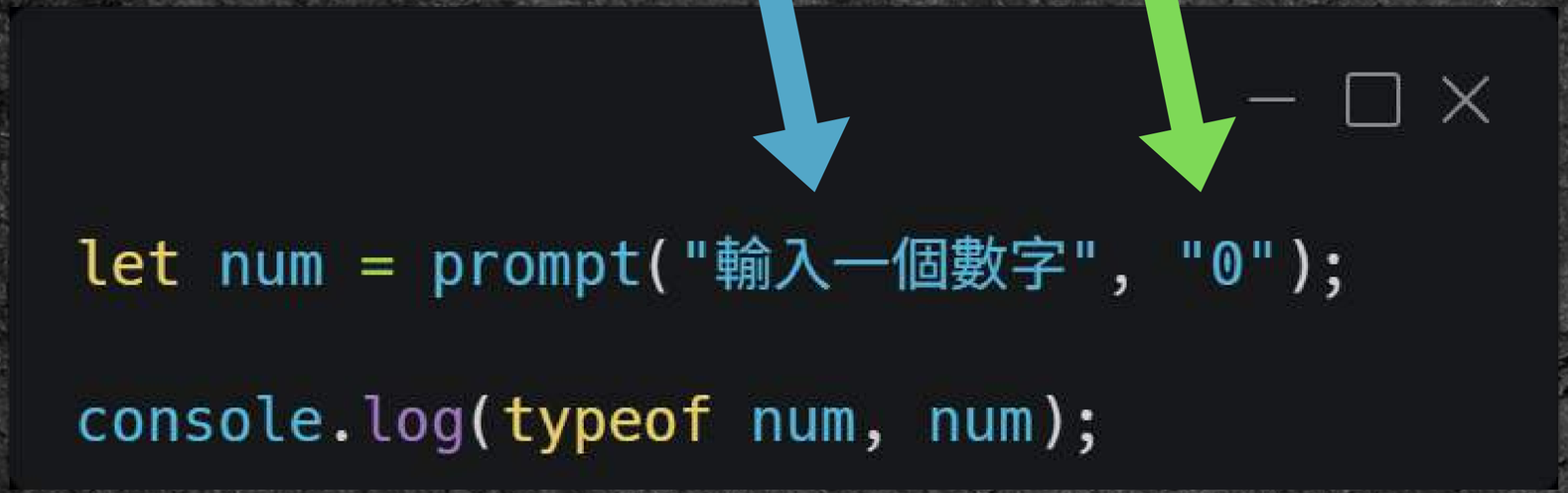
- const 是「只能讀取」的變數，變數宣告後不能再修改

```
const x = 1;
x = 2;
```



輸入：prompt( )；

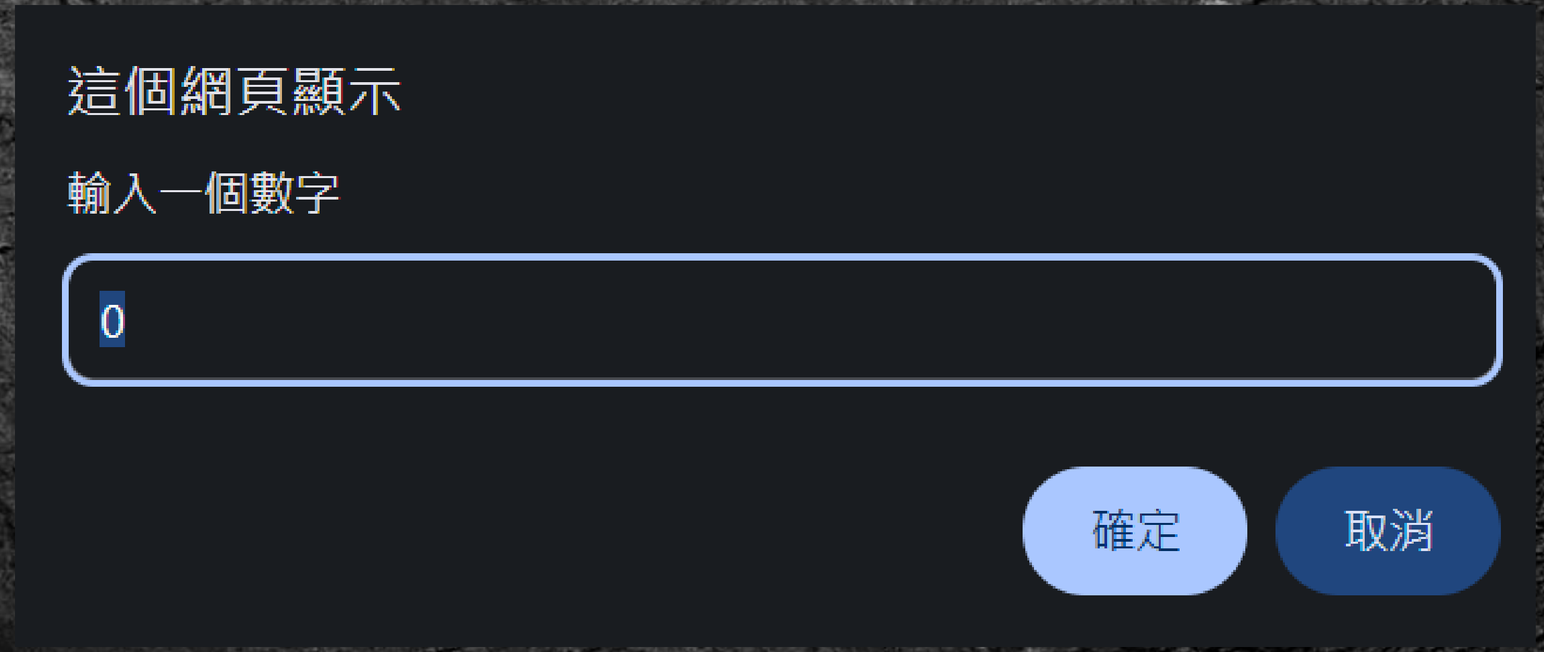
prompt(輸入提示，預設輸入);



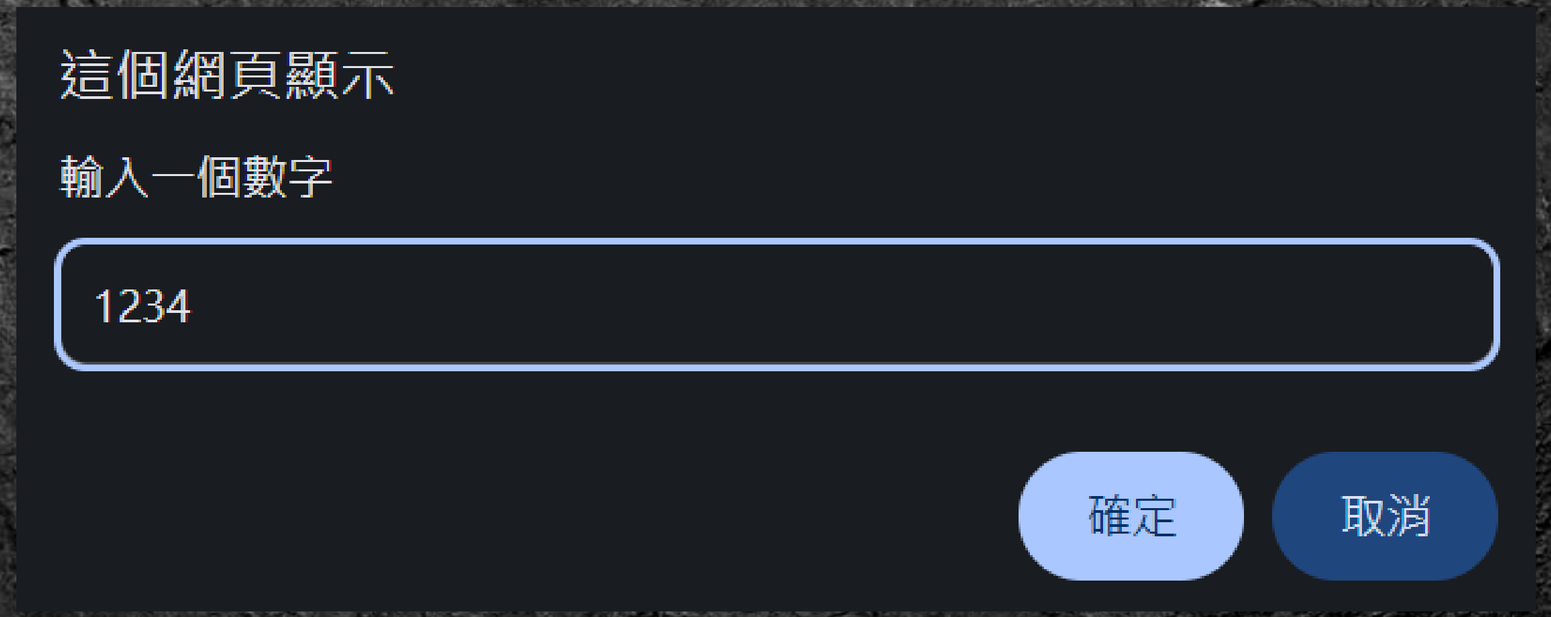
```
let num = prompt("輸入一個數字", "0");

console.log(typeof num, num);
```

```
let num = prompt("輸入一個數字", "0");

console.log(typeof num, num);
```

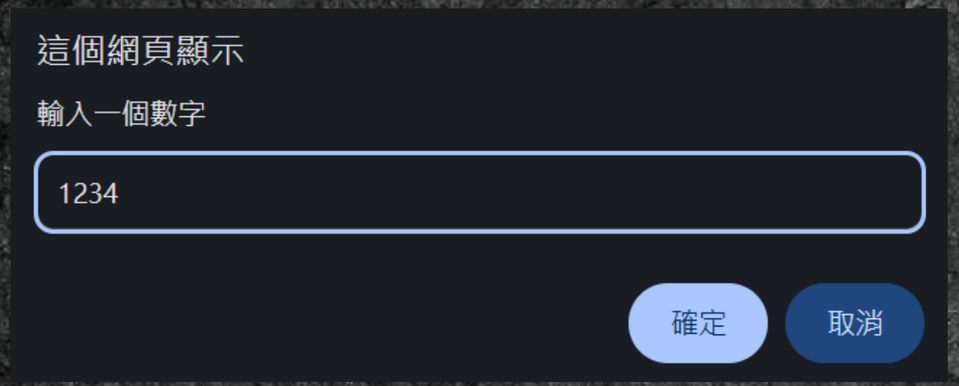
- 輸入視窗

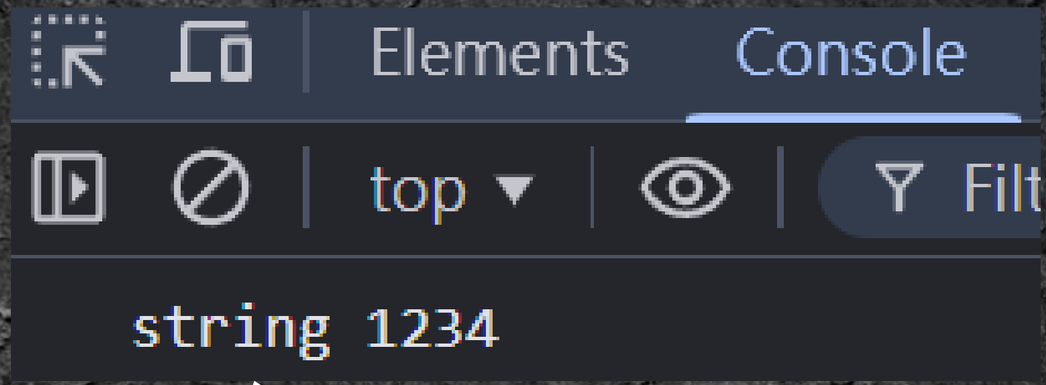


```
let num = prompt("輸入一個數字", "0");

console.log(typeof num, num);
```



```
let num = prompt("輸入一個數字", "0");

console.log(typeof num, num);
```





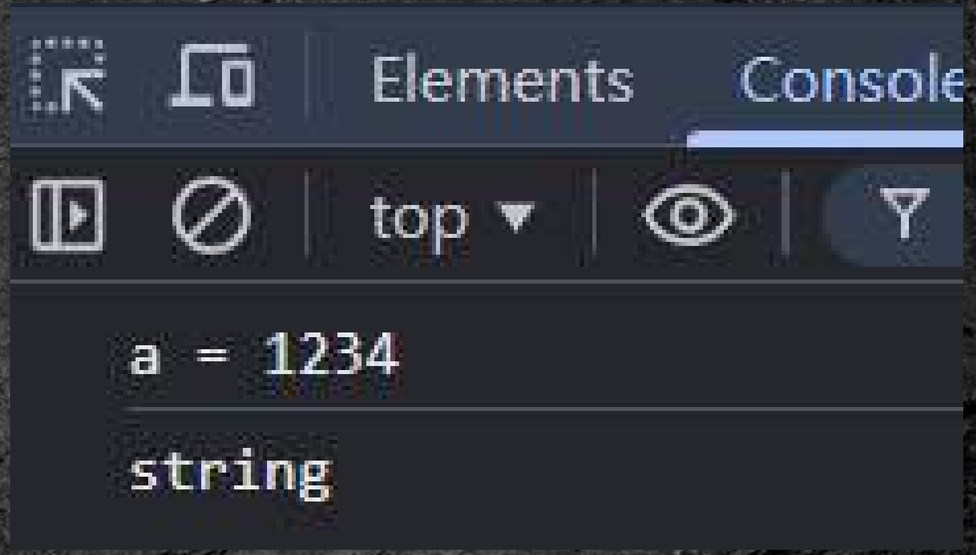
- **prompt 會將資料以「字串」的型態儲存**

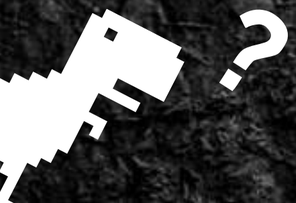
輸入：prompt( ) ;

```
let a = prompt();

a = a + 4;

console.log("a =",a);
console.log(typeof a);
```

輸入：prompt( ) ;

```
let a = prompt();

a = a + 4;

console.log("a =",a);
console.log(typeof a);
```





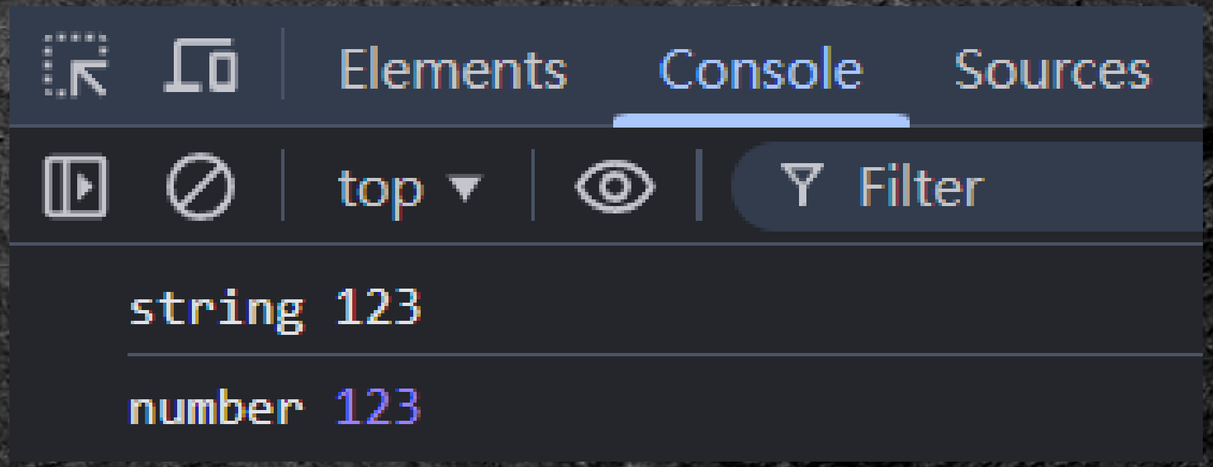
我們要如何輸入「數字」？

Number();

- Number(); 將資料的型態轉為 number

```
let a = prompt();
console.log(typeof a, a);
a = Number(a);
console.log(typeof a, a);
```

```
let a = prompt();
console.log(typeof a, a);
a = Number(a);
console.log(typeof a, a);
```

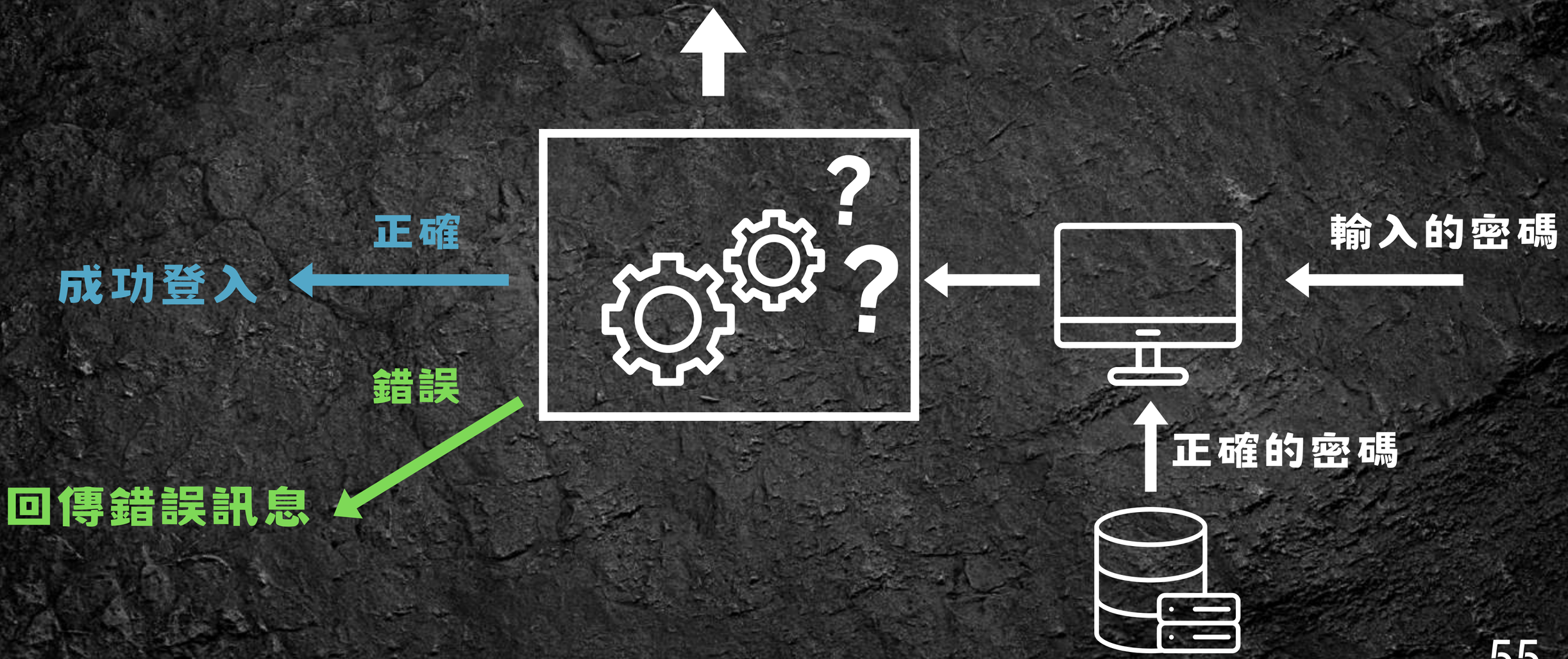


Task 02（5分鐘）

- 輸入 a , b 兩個儲存數字的變數
- 嘗試用 a , b 去做數學運算（+, -, *, /, **, %, ()）
- let a = ? ;
- prompt( ) ;
- Number();

條件判斷式 與 比較運算

如何判斷是不是正確的密碼，並做出回應？



正確

成功登入

錯誤

回傳錯誤訊息

條件判斷式

小提醒：

真正的網頁不會讓帳號和密碼出現在前端

比較運算

- 比較兩邊，運算結果會回傳布林值（true, false）

大於 > 、小於 <
大於等於 >=
小於等於 <=
(不)等於 == , !=
嚴格(不)等於 === ,!==

- 建議使用 === 和 !==

比較運算

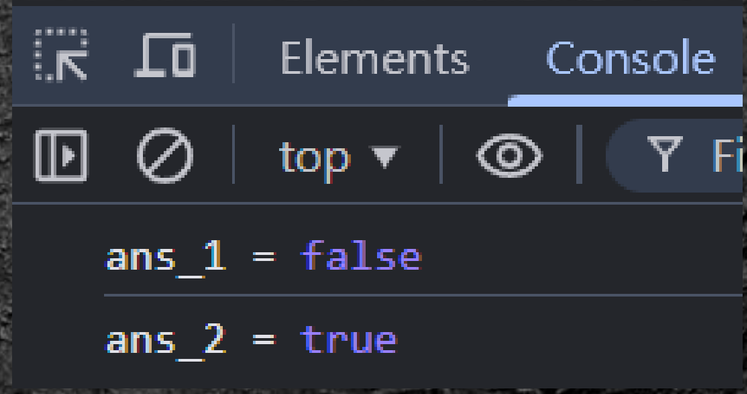
大於 > 、小於 <
大於等於 >=
小於等於 <=
(不)等於 == , !=
嚴格(不)等於 === ,!==

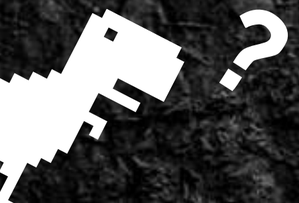
```
let a = 2, b = 5;

let ans_1 = (a > b);

let ans_2 = (a !== b);

console.log("ans_1 =", ans_1);
console.log("ans_2 =", ans_2);
```

比較運算

大於 > 、小於 <
大於等於 >=
小於等於 <=
(不)等於 == , !=
嚴格(不)等於 === ,!==

```
let a = 2, b = 5;

let ans_1 = (a > b);

let ans_2 = (a !== b);

console.log("ans_1 =", ans_1);
console.log("ans_2 =", ans_2);
```





== VS. ===

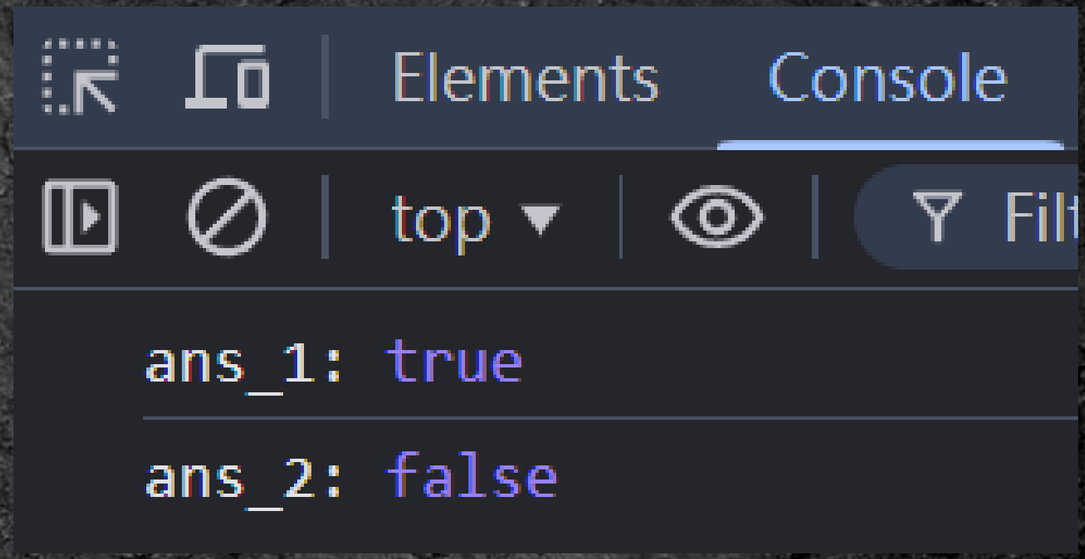
ans_1 = ?, ans_2 = ?

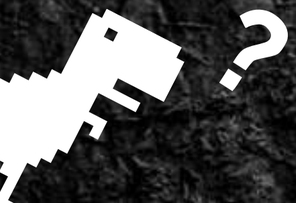
```
let a = "123", b = 123;

let ans_1 = (a == b);

let ans_2 = (a === b);

console.log("ans_1:", ans_1);
console.log("ans_2:", ans_2);
```

== VS. ===

ans_1 = ?, ans_2 = ?

```
let a = "123", b = 123;

let ans_1 = (a == b);

let ans_2 = (a === b);

console.log("ans_1:", ans_1);
console.log("ans_2:", ans_2);
```





if 條件式

- if （如果）
- else （否則）

```
if(條件){
    code_1;
}
else{
    code_2;
}
```

⟵ 如果條件成立執行 code_1，

否則執行 code_2

else if()

```
if(條件_1){
    code_1;
}
else if(條件_2){
    code_2;
}
else{
    code_3;
}
```

- else if （否則如果）
- 可能條件的數量 > 2 時使用
- 如果 if 不成立，會先跳到 else if

else if()

```
if(條件_1){
    code_1;
}
else if(條件_2){
    code_2;
}
else{
    code_3;
}
```

- else if （否則如果）
- 可能條件的數量 > 2 時使用

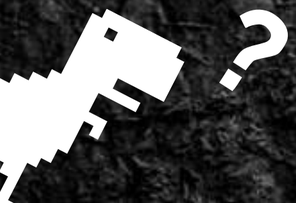
- 如果 if 不成立，會先跳到 else if

- else if 也不成立，才跳到 else

else if()

- else if 可能不只一個

```
let score = Number(prompt());

if(score >= 80){
    console.log("A");
}
else if(score >= 70){
    console.log("B");
}
else if(score >= 60){
    console.log("C");
}
else{
    console.log("Failure!");
}
```



邏輯運算符號

&& AND （且）

a \ b	True	False
True	True	False
False	False	False

|| OR （或）

a \ b	True	False
True	True	True
False	True	False

! NOT(反相)

a	! a
True	False
False	True

邏輯運算: AND （&&）

a \ b	True	False
True	True	False
False	False	False

→ a: BangDream的角色

→ b:主唱 + 吉他

```
if(a && b){
    console.log("name:","戸山香澄")
    console.log("Popipa! Pipopa! Popipapapipopa!");
}
```

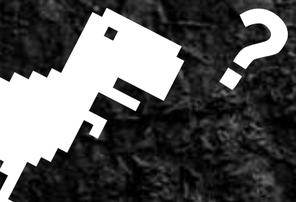
邏輯運算: OR（||）

JavaScript 入門

主題課程 2

a \ b	True	False
True	True	True
False	True	False

```
let CLass = "JavaScript";
let Type = "主題課程_2";

if(CLass === "JavaScript" || Type === "主題課程_2"){
    console.log("講師:", "鴕鳥");
}
```



巢狀 if

- if() 裡又有 if()，一層一層的 if 就是巢狀 if()

```
if(條件_1){
    if(條件_2){
        if(條件_3){

        }
    }
}
```

```
if(Type === "主題課程"){
    if(CLass === "JavaScript"){
        console.log("鴕鳥");
    }
    else if(CLass === "HTML & CSS"){
        console.log("豬肉切");
    }
}
else{
    console.log("通識課程");
}
```

Task 03（10分鐘）

- 輸入兩個數字 a , b 比較兩個數字的大小並輸出
  1. a > b
  2. a = b
  3. a < b 其中一種答案
- if( ) , else if( ) , else

迴圈（loop）

如果要在 Console 上輸出 100 個 UMU 該怎麼做?

放法 1： 複製 + 貼上暴力法

```
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
.
.
.
```

暴力法好用，但我們有更好的解法

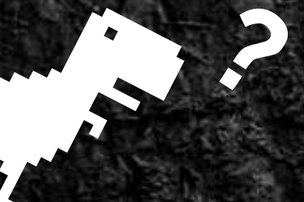
放法 2：迴圈 :D

```
for(let i = 0 ; i < 100 ; i += 1){
    console.log("UMU");
}
```

為什麼需要迴圈?

- 簡化我們程式碼
- 整合程式中「重複」的操作

```
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
console.log("UMU");
.
.
.
```



```
for(let i = 0 ; i < 100 ; i += 1){
    console.log("UMU");
}
```

while 迴圈（while loop）

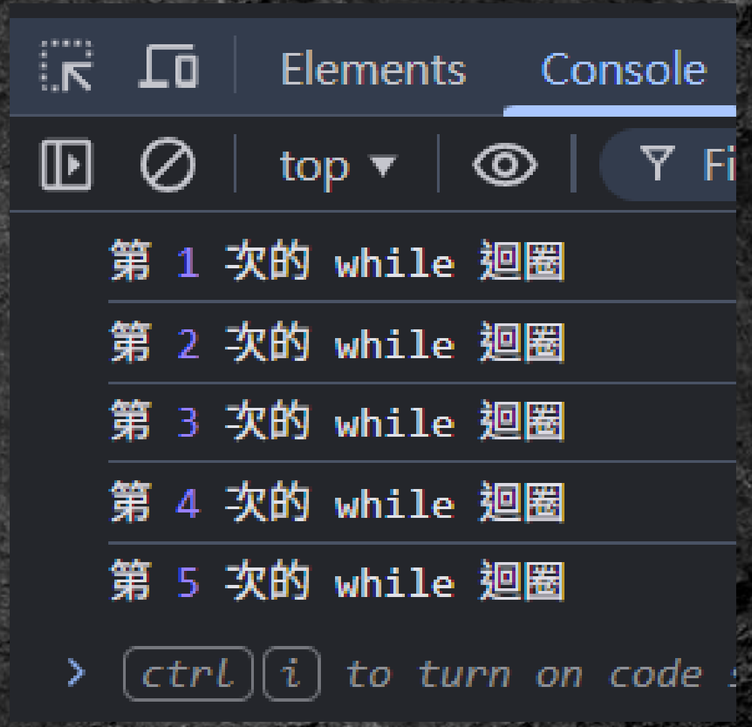
```
while(條件){
    code;
}
```

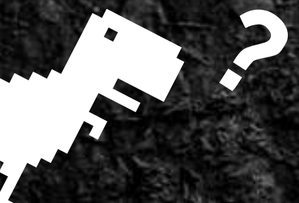
先判斷再執行

- 先進到條件判斷，再執行迴圈的程式
- 只要條件不成立就會離開迴圈

while 迴圈（while loop）

```
let n = 0;
//跑 5 次 while 迴圈
while(n < 5){
    n += 1;
    console.log("第", n, "次的 while 迴圈");
}
```





無限迴圈

- 無限迴圈 → 跑一輩子也**無法離開**的迴圈

無限迴圈

- 無限迴圈 → 跑一輩子也**無法離開**的迴圈

無限迴圈

```
let n = 0;
let i = 0;
while(n < 10){
    i += 1;
    console.log("i =",i);
}
```

- 註：無限迴圈可能導致網頁卡死，不要輕易嘗試

無限迴圈

```
let n = 0;
let i = 0;
while(n < 10){
    i += 1;
    console.log("i =",i);
}
```

- n 的值永遠是 0，不會被修改，因此我們無法離開迴圈

- 註：無限迴圈可能導致網頁卡死，不要輕易嘗試

for 迴圈

```
for( A ; B ; C )
{
  code
}
```

- A: 第一次進入 for 迴圈時執行的動作

for 迴圈

```
for( A ; B ; C )
{
  code
}
```

- A: 第一次進入 for 迴圈時執行的動作

- B: for 迴圈的條件判斷

for 迴圈

```
for( A ; B ; C )
{
  code
}
```

- A: 第一次進入 for 迴圈時執行的動作
- B: for 迴圈的條件判斷

條件成立，執行 code

- C: 跑完 code 後執行的程式

for 迴圈

```
for( A ; B ; C )
{
  code
}
```

- A: 第一次進入 for 迴圈時執行的動作
- B: for 迴圈的條件判斷

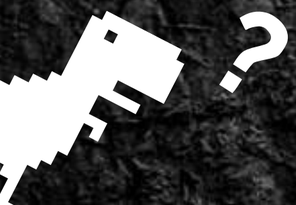
條件成立，執行 code

- C: 跑完 code 後執行的程式

for 迴圈

```
for( A ; B ; C )
{
  code
}
```

- A: 第一次進入 for 迴圈時執行的動作
- B: 檢查執行 code 的條件
- 條件不成立
- 離開 for 迴圈



一個跑 100 次的 for 迴圈，每次的迴圈都會輸出一行 "UMU"

```
let cnt = 0; //計算迴圈執行次數

for(cnt = 0 ; cnt < 100 ; cnt += 1){
    console.log("UMU");
}
```

```
let cnt = 0; //計算迴圈執行次數

for(cnt = 0 ; cnt < 100 ; cnt += 1){
    console.log("UMU");
}
```

- A：第一次進到 for 迴圈，把 cnt 的值重設為 0

```
let cnt = 0; //計算迴圈執行次數

for(cnt = 0 ; cnt < 100 ; cnt += 1){
    console.log("UMU");
}
```

- B：for 迴圈的執行條件，檢查 for 迴圈已經跑過幾次?

```
let cnt = 0; //計算迴圈執行次數

for(cnt = 0 ; cnt < 100 ; cnt += 1){
    console.log("UMU");
}
```

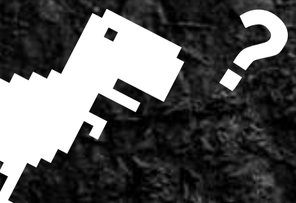
- code： 在 Console 上輸出 "UMU"

```
let cnt = 0; //計算迴圈執行次數

for(cnt = 0 ; cnt < 100 ; cnt += 1){
    console.log("UMU");
}
```

- C：修改 cnt 的值 → 重新紀錄 for 迴圈的執行次數

```
let cnt = 0; //計算迴圈執行次數

for(cnt = 0 ; cnt < 100 ; cnt += 1){
    console.log("UMU");
}
```

- C → B：執行完 C 部分我們會回到 B，檢查要不要繼續跑 for 迴圈



變數的有效範圍

- i_1 和 i_2 兩種寫法差別在哪?

```
<script>

    let i_1 = 0;
    for(i_1 = 0 ; i_1 < 5; i_1 += 1){
        console.log("i_1 =", i_1);
    }


    for(let i_2 = 0 ; i_2 < 5; i_2 += 1){
        console.log("i_2 =", i_2);
    }

</script>
```

變數的有效範圍

```
<script>

    let i_1 = 0;
    for(i_1 = 0 ; i_1 < 5; i_1 += 1){
        console.log("i_1 =", i_1);
    }


    for(let i_2 = 0 ; i_2 < 5; i_2 += 1){
        console.log("i_2 =", i_2);
    }

</script>
```

- 在整個 <script> 都可以使用

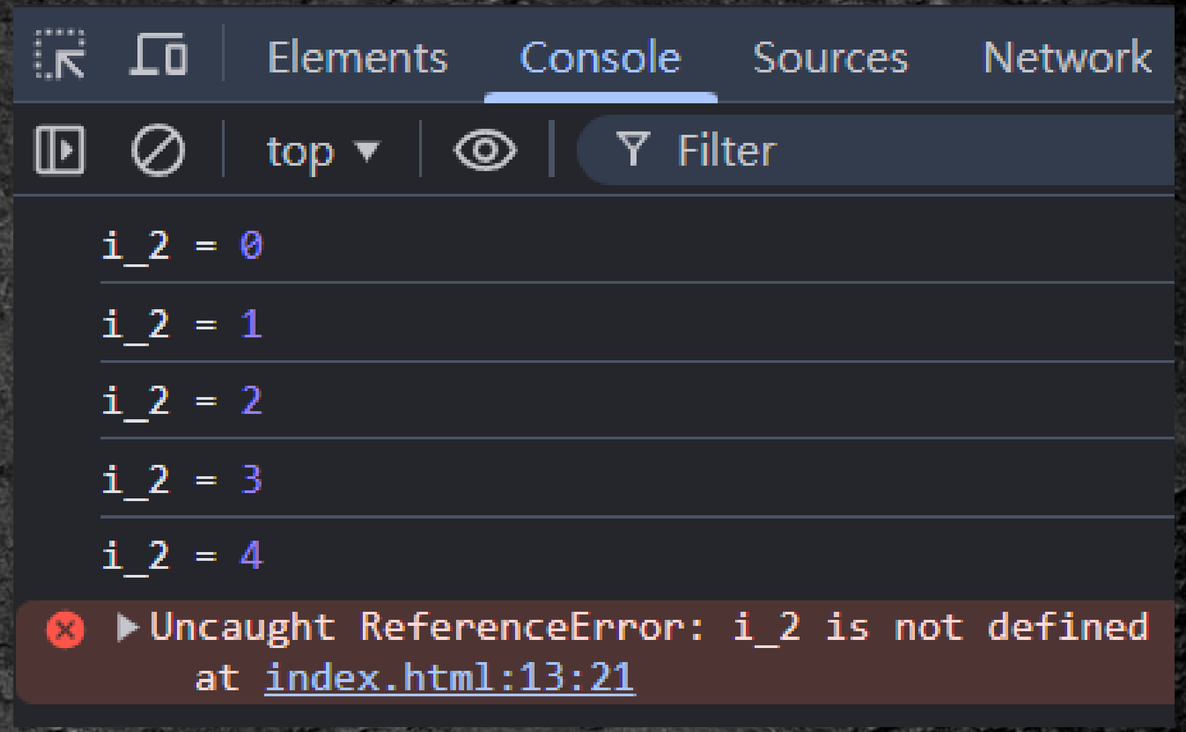
變數的有效範圍

```
<script>

    let i_1 = 0;
    for(i_1 = 0 ; i_1 < 5; i_1 += 1){
        console.log("i_1 =", i_1);
    }


    for(let i_2 = 0 ; i_2 < 5; i_2 += 1){
        console.log("i_2 =", i_2);
    }

</script>
```

- 因為在 for 迴圈裡宣告變數，這個變數只在 for 迴圈中使用

變數的有效範圍

```
for(let i_2 = 0 ; i_2 < 5; i_2 += 1){
    console.log("i_2 =", i_2);
}

console.log(i_2);
```

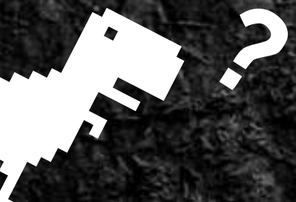


- 離開 for 迴圈 i_2 就會消失

變數的有效範圍

```
for(let i = 0; i < 5 ; i += 1){
    a += 1;
}

for(let i = 0; i < 10 ; i += 1){
    b += 1;
}
```

- 兩個 i 是不同的變數



迴圈控制

迴圈控制讓我們可以中途離開，或中斷執行中的迴圈

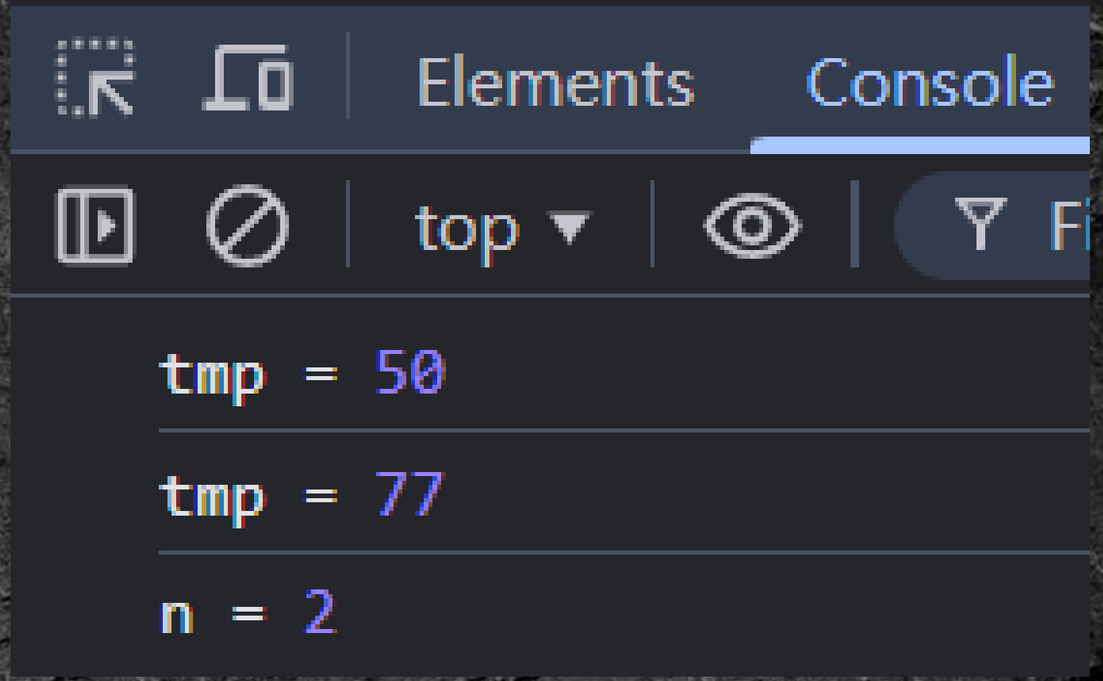
- break; 強制離開迴圈
- continue; 中斷這次的迴圈，直接執行下一個迴圈

迴圈控制 • break; 強制離開迴圈

```
let n = 0;
const ans = 77;

while(n < 6){
    n += 1;//紀錄目前猜的次數

    //輸入猜的答案
    let tmp = Number(prompt());
    console.log("tmp =", tmp);

    if(tmp === ans){
        break;
    }

}

console.log("n =", n);
```

迴圈控制 • break; 強制離開迴圈

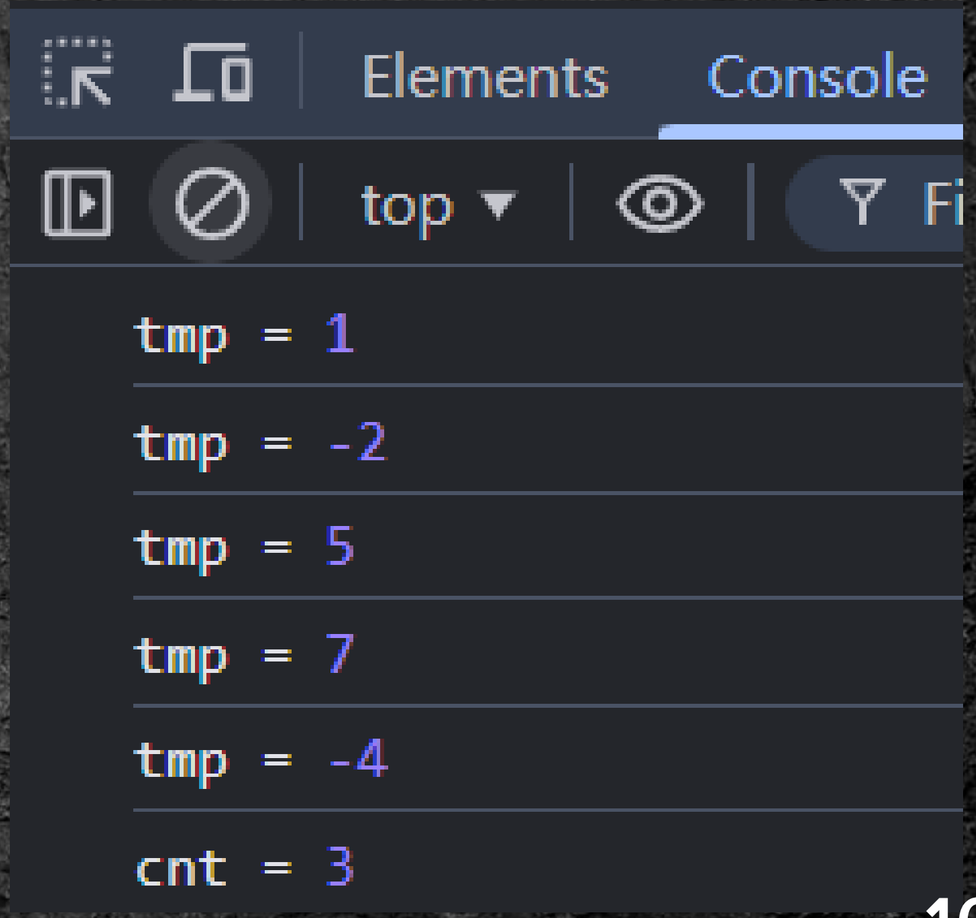
```
let n = 0;
const ans = 77;

while(n < 6){
    n += 1;//紀錄目前猜的次數

    //輸入猜的答案
    let tmp = Number(prompt());
    console.log("tmp =", tmp);

    if(tmp === ans){
        break;
    }

}

console.log("n =", n);
```

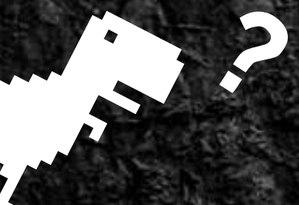


迴圈控制

- continue; 中斷執行中的迴圈， 跳到下一次的迴圈

```
for(let i = 0 ; i < 5 ; i += 1){
    let tmp = Number(prompt());
    console.log("tmp =", tmp);

    if(tmp < 0){
        continue;
    }

    cnt += 1;//計算輸入是正數的數量
}

console.log("cnt =", cnt);
```





Task 04 （10分鐘）

猜數字遊戲: 玩家有最多 6 次的機會來猜答案，答案會是 1 ~ 25 其中一個整數。

- 每次輸入一個數字，並告訴玩家的猜測是大於 或小於 正確答案
- 如果猜了 6 次都沒找到正確答案，或猜到正確答案遊戲都會結束

- 備註： 答案預設一個固定的數字就好

下課 10 分鐘~

陣列（Array）

假設要設計一個餐廳的網頁，我們要怎麼紀錄一連串的食物資訊？

```
let food_1 = "hamburger";
let food_2 = "sandwich";
let food_3 = "bread";
.
.
.
let food_n = "cake";
```

假設要設計一個餐廳的網頁，我們要怎麼紀錄一連串的食物資訊？

```
let food_1 = "hamburger";
let food_2 = "sandwich";
let food_3 = "bread";
.
.
.
let food_n = "cake";
```

我們要面對的問題：

- 如何快速新增或刪除資料？
- 要改菜單的名稱怎麼辦？

  e.g. food → menu

因為工程師是一群懶惰的生物，所以出現了一個新工具讓事情變簡單 → 陣列（Array）

陣列的用途

- 將多個同類型的資料儲存在一起
- 可以快速新增、刪除資料

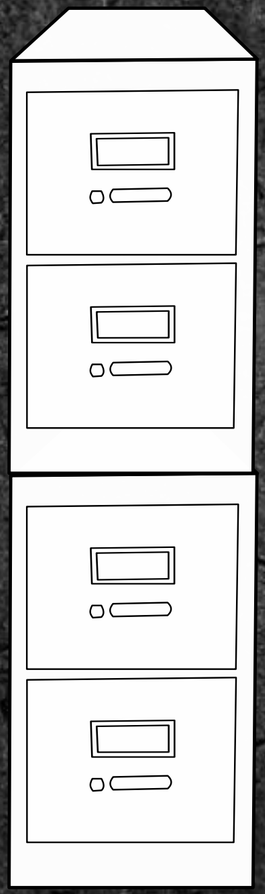
```
let food = ["hamburger", "sandwich", "bread", "cake"];

let num = [0, 1, 2, 3];
```

- 字串陣列
- 數字陣列

- 數字陣列 arr →

```
let arr = [123, 4, 98, 58];
```

抽屜編號	儲存的數字
0	123
1	4
2	98
3	58



- 數字陣列 arr →

```
let arr = [123, 4, 98, 58];
```

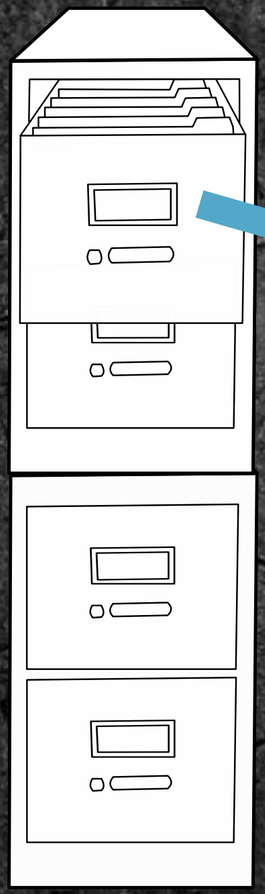
抽屜編號

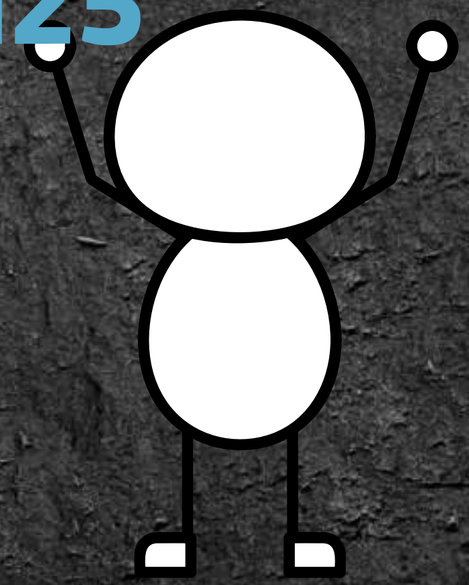
0

1

2

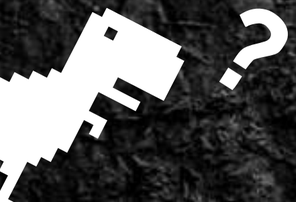
3





拉開編號 0 的抽屜

拿到第一個數字： 123



陣列宣告

```
let Array1 = [];
let Array2 = [0 , 1 , 2 ,3];
```

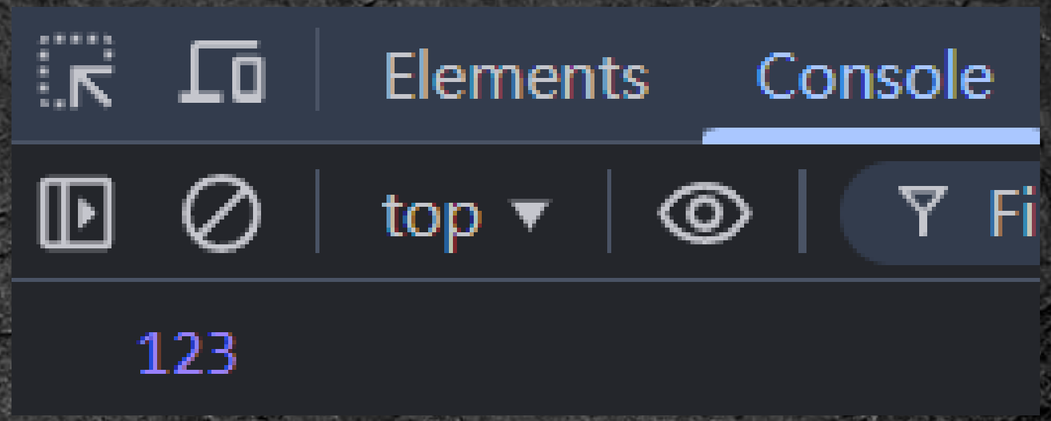
- 宣告空陣列
- 有初始資料的陣列

陣列存取

索引值	0	1	2
資料	123	4	98

- 索引值（index）從 0 開始編號

```
let arr = [123, 4, 98];

let num = arr[0];
console.log(num);
```

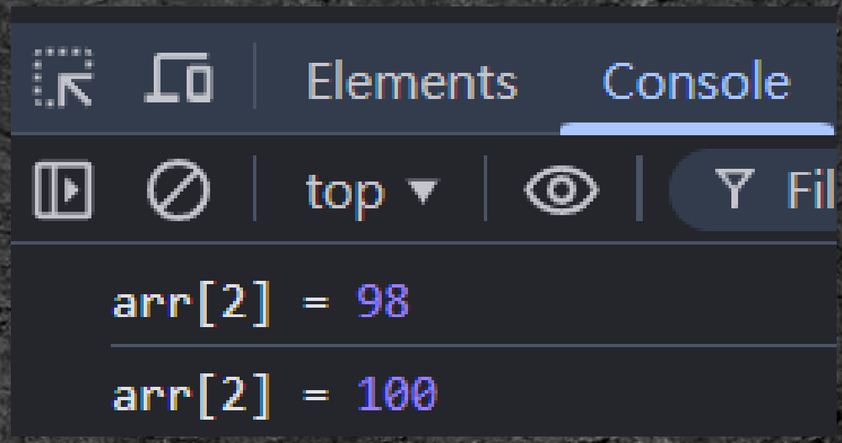


- 取出 arr 的第 1 個數字

陣列存取

索引值	0	1	2
資料	123	4	~~98~~ 100

```
let arr = [123, 4, 98];

console.log("arr[2] =", arr[2]);

arr[2] = 100;

console.log("arr[2] =", arr[2]);
```

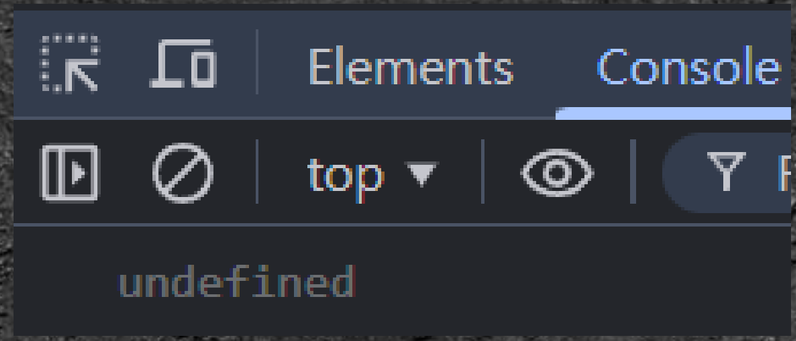


- 取出 arr 的第 2 個數字改成 100

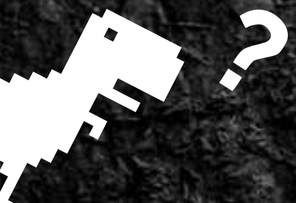
陣列存取

索引值	0	1	2	3
資料	123	4	98	???

```
let arr = [123, 4, 98];

console.log(arr[3]);
```



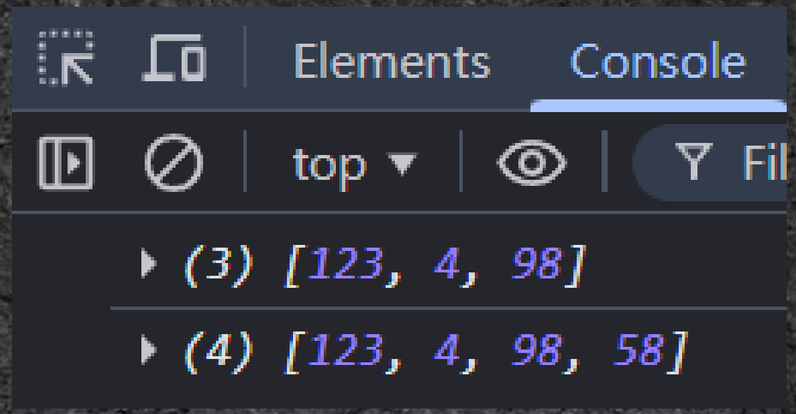
- 存取超過陣列的範圍



新增資料

- push(); 將資料新增（推入） 到陣列的最後面

```
let arr = [123, 4, 98];
console.log(arr);

arr.push(58);
console.log(arr);
```

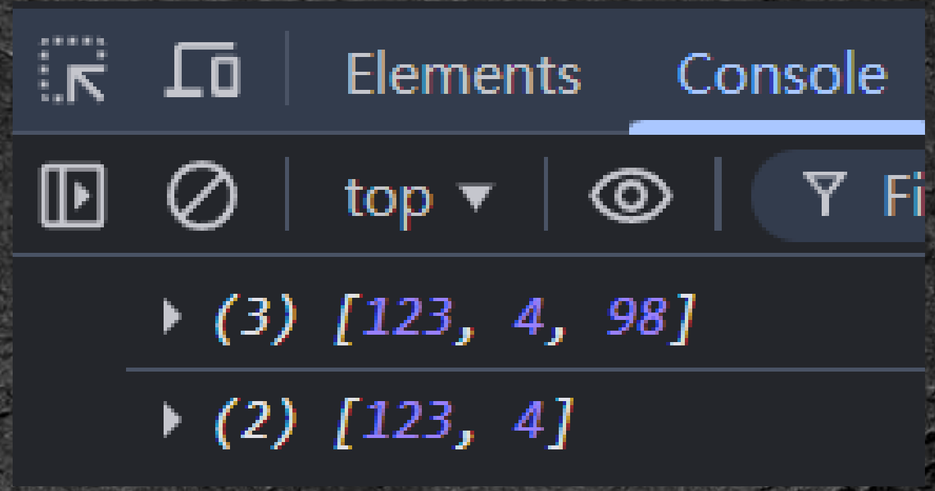


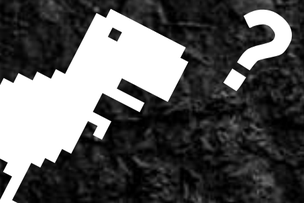
- 新增 58 到 arr 中

刪除資料

- pop(); 刪除陣列的最後一個元素

```
let arr = [123, 4, 98];
console.log(arr);

arr.pop();
console.log(arr);
```





陣列搜尋：indexOf()

- 找目標在陣列中的 index = ？

```
let Array = ['a' , 'b' , 'c' , 'a'];

let index = Array.indexOf('a');
console.log("index =",index);

index = Array.indexOf('a' , 1);
console.log("index =",index);
```

indexOf(item, start);

預設為 0

- 找 'a' 在 Array 中的 index = ？

陣列搜尋：indexOf()

- 找目標在陣列中的 index = ？

```
let Array = ['a' , 'b' , 'c' , 'a'];

let index = Array.indexOf('a');
console.log("index =",index);

index = Array.indexOf('a' , 1);
console.log("index =",index);
```

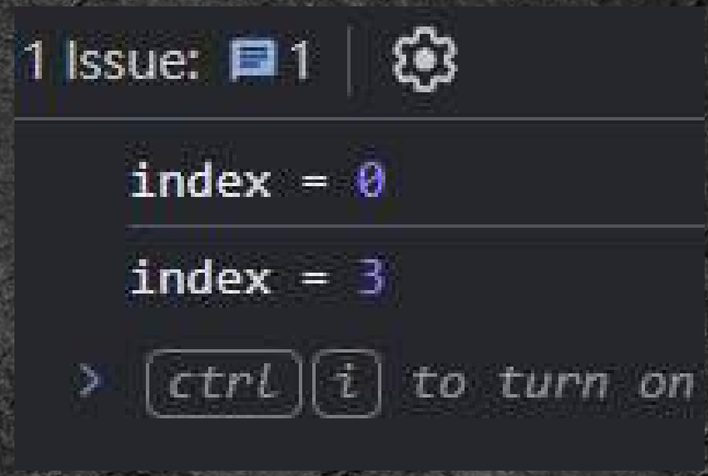
indexOf(item, start);

預設為 0

- 從 index = 1 的位置開始搜尋

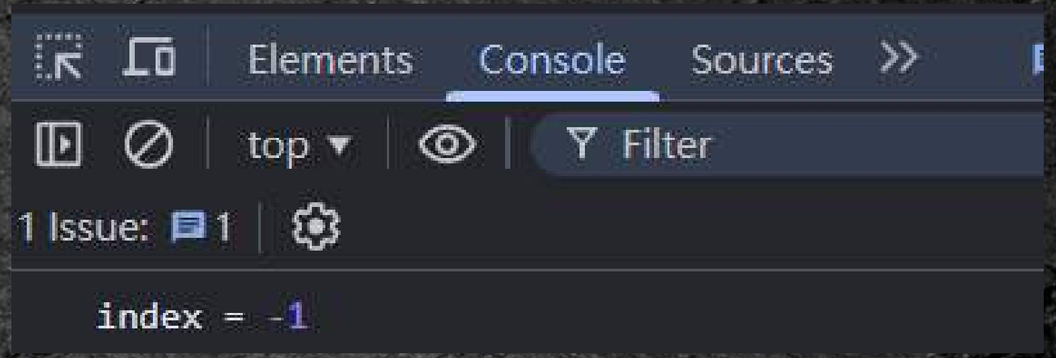
陣列搜尋：indexOf()

```
let Array = ['a' , 'b' , 'c' , 'a'];

let index = Array.indexOf('a');
console.log("index =",index);

index = Array.indexOf('a' , 1);
console.log("index =",index);
```



陣列搜尋：indexOf()

```
let Array = ['a' , 'b' , 'c' , 'a'];

let index = Array.indexOf('D');
console.log("index =",index);
```



- 沒找到會回傳 -1

陣列搜尋：includes()

- 看目標有沒有在出現陣列中

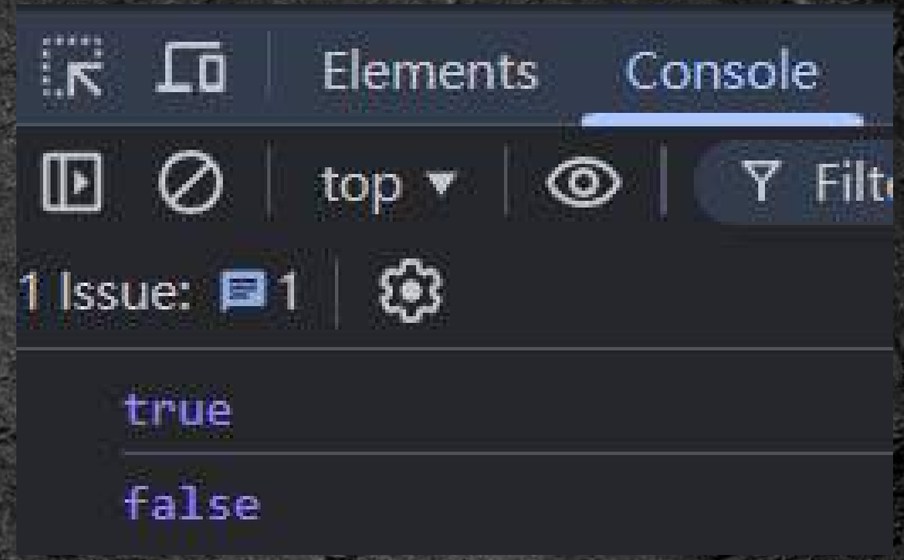
```
let Array = ['a' , 'b' , 'c' , 'a'];

let test = Array.includes('b');
console.log(test);

test = Array.includes('D');
console.log(test);
```

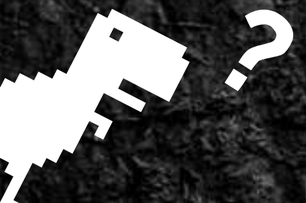
includes(item);

- 找到回傳 true
- 沒找到回傳 false

陣列搜尋：includes()

```
let Array = ['a' , 'b' , 'c' , 'a'];

let test = Array.includes('b');
console.log(test);

test = Array.includes('D');
console.log(test);
```





其他陣列的相關知識

Task 05 （10 分鐘）

- 用一個陣列 A 儲存 n 個數字
- 計算陣列中所有數字的總和
- push();
- for 迴圈、 while 迴圈

函式 （Function）

什麼是函式（Function）？

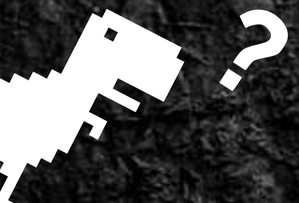
錢（參數）　　販賣機（函式）　　飲料（回傳的資料）

什麼是函式（Function）？

- 通常會把常用的特殊功能打包成函式

我們看過的一些函式

```
// console 上的輸出
log();

// 輸入
prompt();

// 資料型態轉換
Number();

// 陣列
push();
pop();
indexOf();
includes();
```



定義函式

```
function 函式名稱(參數列){
    code;
    return 要回傳的資料;
}
```

```
function add(a, b){//計算 a , b 兩個數字變數相加的結果
    const ans = a + b;
    return ans;
}
```

定義函式

```
function 函式名稱(參數列){
    code;
    return 要回傳的資料;
}
```

```
function add(a, b){//計算 a , b 兩個數字變數相加的結果
    const ans = a + b;
    return ans;
}
```

- 參數： 要做計算的兩個變數 a , b

定義函式

```
function 函式名稱(參數列){
    code;
    return 要回傳的資料;
}
```

```
function add(a, b){//計算 a , b 兩個數字變數相加的結果
    const ans = a + b;
    return ans;
}
```

- 函式的內部程式：計算 a + b 後，把答案丟給變數 ans

定義函式

```
function 函式名稱(參數列){
    code;
    return 要回傳的資料;
}
```

```
function add(a, b){//計算 a , b 兩個數字變數相加的結果
    const ans = a + b;
    return ans;
}
```

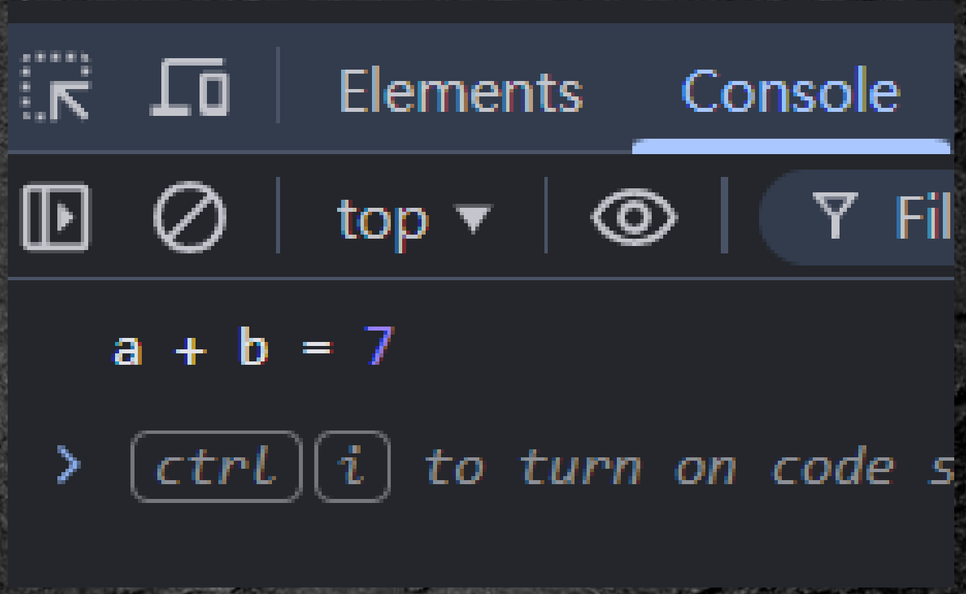
- 函式的回傳： ans 的值 （a + b 的答案）

函式呼叫

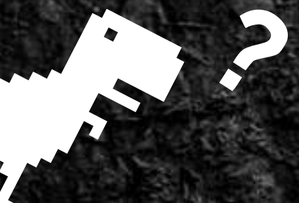
- 使用函式 → 函式呼叫

```
function add(a, b){//計算 a , b 兩個數字變數相加的結果
    const ans = a + b;
    return ans;
}

let a = 2, b = 5;

let ans = add(a, b);
console.log("a + b =", ans);
```

呼叫 add()



- 函式要定義後才能呼叫



```
let A = [1, 2, 3], len_A = 3;
let B = [2, 4, 6], len_B = 3;
let target = 1;

let ans = false;
for(let i = 0; i < len_A; i++){
    if(target === A[i]){
        ans = true;
        break;
    }
}
console.log(ans);

ans = false;
for(let i = 0; i < len_B; i++){
    if(target === B[i]){
        ans = true;
        break;
    }
}
console.log(ans);
```

```
function my_includes(target, arr, len){
    let ans = false;

    for(let i = 0 ; i < len ; i+= 1){
        if(target === arr[i]){
            ans = true;
            break;
        }
    }

    console.log(ans);
}

let A = [1 ,2 ,3];
let B = [2, 4, 6];

my_includes(1, A, 3);
my_includes(1, B, 3);
```

- 函式可以重複利用
- 簡化程式碼

函式不一定需要有回傳值

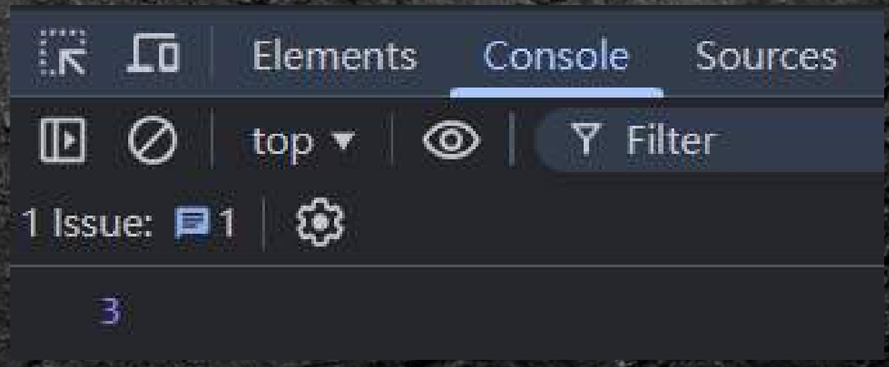
```
function Print(){
    console.log("ouob");
    return;
}
```

Task 06 （8分鐘）

寫一個函式 Max(a , b);

- 功能: 比較兩個數字的大小， 回傳比較大的那個數字，
  如果一樣大就回傳任意一個
- 參數: 變數 a , b
- if , else if , else

```
let a = 3 , b = 2;
console.log(Max(a,b));
```



物件（Object）

假設今天要自我介紹，你會怎麼介紹自己?

- 名字：鴕鳥
- 大學的科系：資工系
- 身高：180
- 喜歡的樂團：Poppin'Party

人

- 名字：鴕鳥
- 大學的科系：資工系
- 身高：180
- 喜歡的樂團：Poppin'Party

人 = 一個物件（Object）

- 名字：鴕鳥
- 大學的科系：資工系
- 身高：180
- 喜歡的樂團：Poppin'Party

物件的屬性（Property）

一個人可能會做那些事情？

人

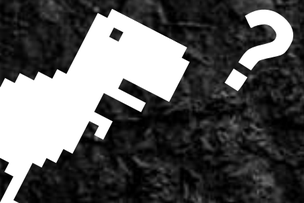
- 睡覺
- 吃飯
- 打球

一個人

- 睡覺： Sleep();
- 吃飯： Eat();
- 打球： Play_ball();

物件的特殊功能 （函式）

物件的方法（Method）



資工系是一個好地方，對吧？

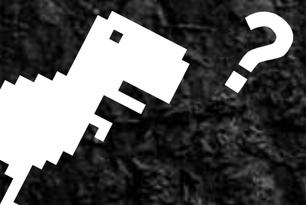
物件（Object）的用途

- 將多個**有關聯的**變數、陣列、函式打包成物件

```
name = "Ostrich";
major = "CSIE";
Height = 180;

function Sleep(){
    onsole.log("ZZZ...");
}
function Eat(){
    //eat something
}
```

打包成物件

```
const Human = {
    name: "Ostrich",
    major: "CSIE",
    Height: 180,

    Sleep: function(){
        console.log("ZZZ...");
    },
    Eat: function(){
        //eat something
    }
}
```



建立物件 → JSON（JavaScript Object Notation）

JavaScript物件表示法

```
const Player = {
    name: "Bob",
    level: 26,
    data: [100, 50],//[HP, MP]

    Print: function(){
        console.log("ouob");
    }
}
```

建立物件（JSON）

- 物件的名稱

```
const Player = {
    name: "Bob",
    level: 26,
    data: [100, 50],//[HP, MP]

    Print: function(){
        console.log("ouob");
    }
}
```

建立物件（JSON）

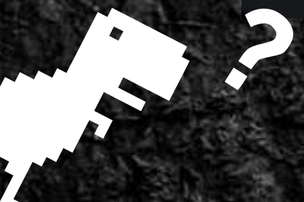
```
const Player = {
    name: "Bob",
    level: 26,
    data: [100, 50],//[HP, MP]

    Print: function(){
        console.log("ouob");
    }
}
```

- 物件的屬性（Property）：
  - 變數： name 、 level
  - 陣列： data
- 物件的方法（Method）：
  - 函式： Print();

建立物件（JSON）

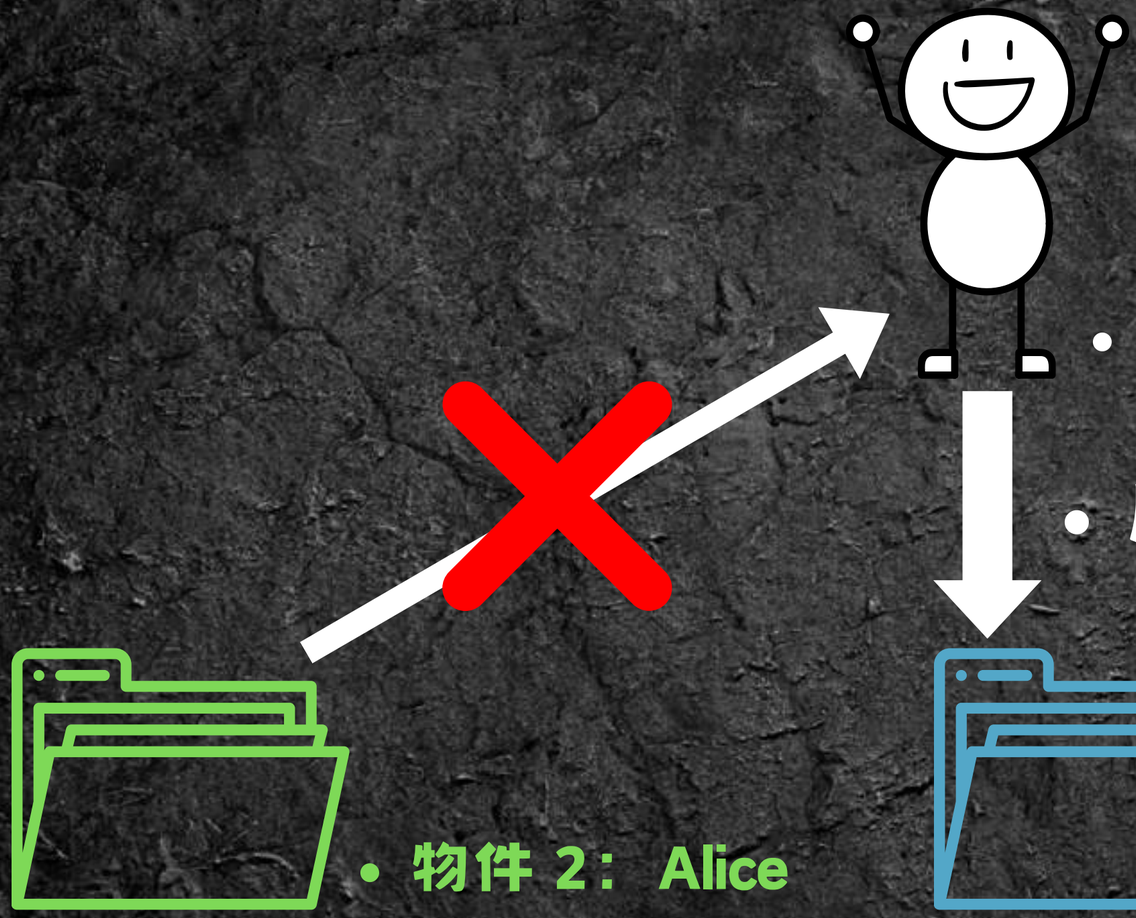
```
const Player = {
    name: "Bob",
    level: 26,
    data: [100, 50],//[HP, MP]

    Print: function(){
        console.log("ouob");
    }
}
```

- 用逗號隔開物件的變數、函式



為什麼要用 const？

```
const Player = {
    name: "Bob",
    level: 26,
    data: [100, 50],//[HP, MP]

    Print: function(){
        console.log("ouob");
    }
}
```



- 物件 Player
- 用 const 固定 Player 指向的物件資料
- 物件 2： Alice
- 物件 1： Bob

為什麼要用 const？

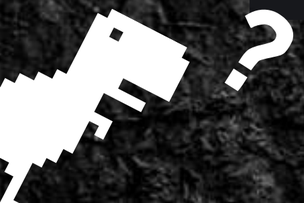
```
const Player = {
    name: "Bob",
    level: 26,
    data: [100, 50],//[HP, MP]

    Print: function(){
        console.log("ouob");
    }
}

Player = {
    name: "Alice",
    level: 999
}
```

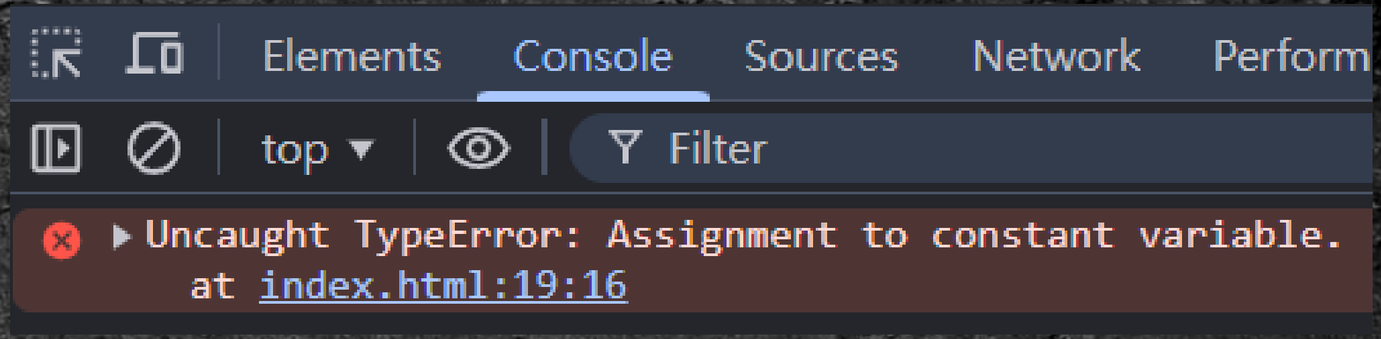
- 物件 1：Bob
- 物件 2：Alice

為什麼要用 const？

```
const Player = {
    name: "Bob",
    level: 26,
    data: [100, 50],//[HP, MP]

    Print: function(){
        console.log("ouob");
    }
}

Player = {
    name: "Alice",
    level: 999
}
```

- 物件 1： Bob
- 物件 2： Alice





- 不能改變 Player 指向的物件

物件的存取

key： name　　物件 Player　　value： "Bob"

```
const Player = {
    name: "Bob",
    level: 26,
    data: [100, 50],//[HP, MP]

    Print: function(){
        console.log("ouob");
    }
}

console.log( Player.name );
```

取出 Player 的 name 屬性

物件的存取


```
const Player = {
    name: "Bob",
    level: 26,
    data: [100, 50],//[HP, MP]

    Print: function(){
        console.log("ouob");
    }
}

console.log( Player.name );
```

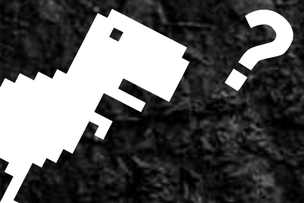
key	value
name	"Bob"
level	26
data	[100, 50]
Print	函式 Print()

物件的存取

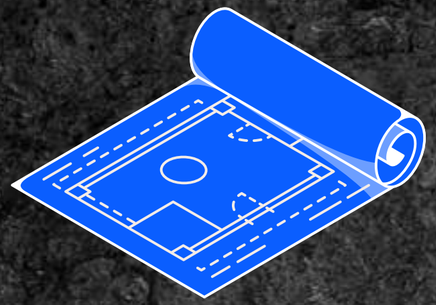

```
const Player = {
    name: "Bob",
    level: 26,
    data: [100, 50],//[HP, MP]

    Print: function(){
        console.log("ouob");
    }
}

//取出 Player 的 name
console.log( Player.name );

Player.level = 100;

Player.Print();
```

改變 Player 的 level 儲存的值

呼叫 Player 的 Print()



複製物件 Object.create();



Player

複製

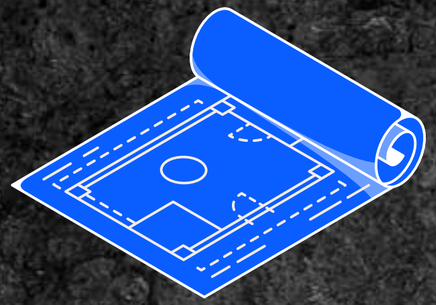
新物件

複製現有的物件資料來建立新的物件

```
const Player = {
    name: "Bob",
    level: 26,
    data: [100, 50]//[HP, MP]
}

const New_obj = Object.create(Player);

console.log( New_obj );
```

- 用 Player 的資料建立新物件

複製物件 Object.create();



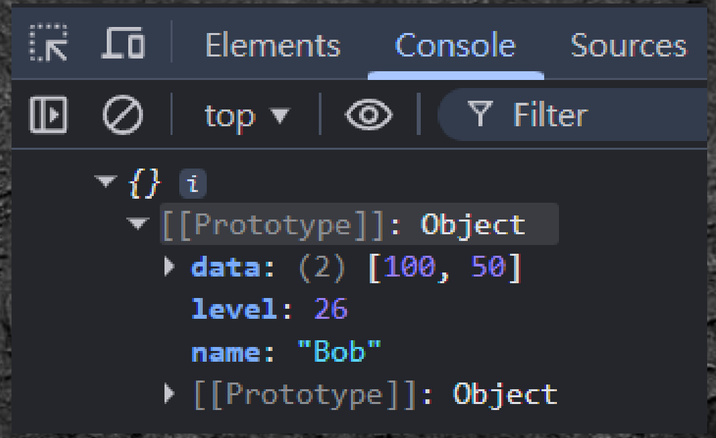
Player

複製

新物件

複製現有的物件資料來建立新的物件

```
const Player = {
    name: "Bob",
    level: 26,
    data: [100, 50]//[HP, MP]
}

const New_obj = Object.create(Player);

console.log( New_obj );
```



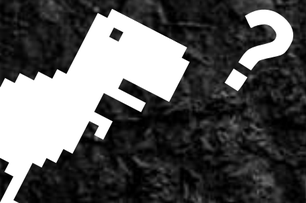
- 用 Player 的資料建立新物件

this → 代替物件自己

```
const Player = {
    name: "Bob",
    level: 26,
    data: [100, 50],//[HP, MP]

    Print: function(){
        console.log("name:",Player.name);
        console.log("level:",Player.level);
    }
}
```

```
const Player = {
    name: "Bob",
    level: 26,
    data: [100, 50],//[HP, MP]

    Print: function(){
        console.log("name:",this.name);
        console.log("level:",this.level);
    }
}
```

- this = Player



Task 07 （10分鐘）

- 寫一個物件 Player 儲存遊戲的玩家資料
- Player 的內容：

  - name: 玩家名字
  - level: 玩家等級
  - data: [力量， 智力]（[STR, INT]）
  - 函式 Power()： 計算戰鬥力並輸出

    戰鬥力 = level ×（STR + INT）

```
function Power(level, STR, INT){
    const PWR = level * (STR + INT);
    console.log(PWR);
}
```

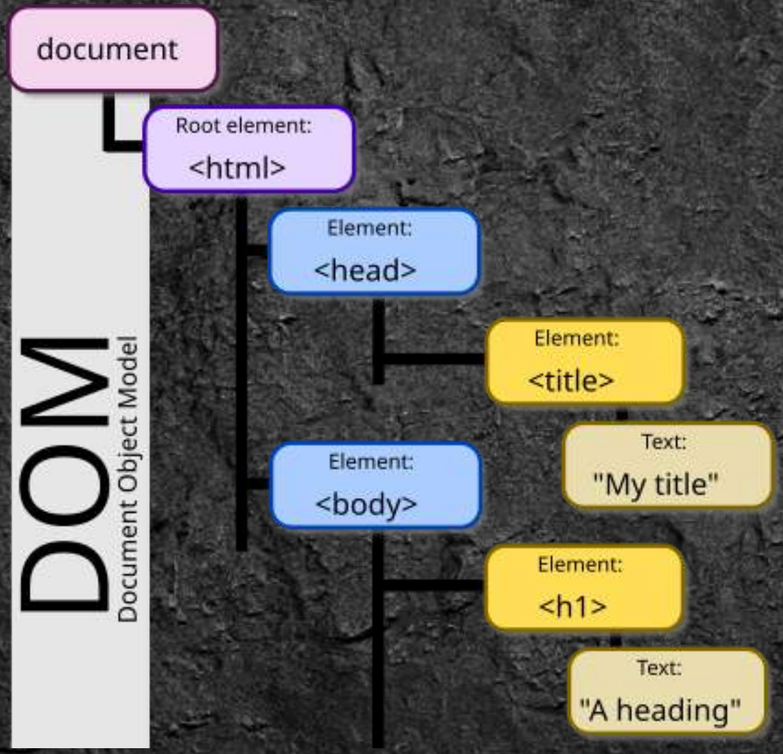
下課 10 分鐘~

HTML DOM

HTML Document Object Model

文件物件模型（Document Object Model）

HTML DOM 讓我們可以用 JavaScript 去取得、修改、新增或刪除 HTML 中的內容




```html
<!DOCTYPE html>
<html>
    <head>

        <title>My title</title>

    </head>

    <body>

        <h1>A heading</h1>

    </body>
</html>
```

HTML 標籤 → JavaScript 物件

```
<body>

    <div id = "ouo" style="font-size: 100px;">
        empty
    </div>

    <script>

        const obj = document.querySelector("#ouo");

        obj.innerHTML = "New content!!";

    </script>

</body>
```

把 obj 指向 id = "ouo" 的標籤物件

物件 obj

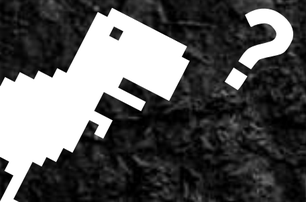
存取、修改

id = "ouo" 的 div 標籤

HTML 標籤 → JavaScript 物件

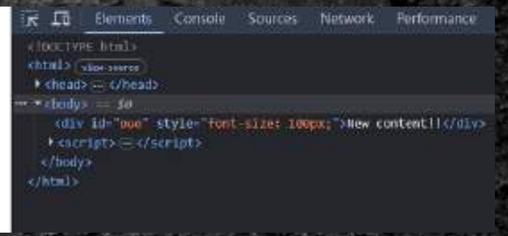
```html
<body>

    <div id = "ouo" style="font-size: 100px;">
        empty
    </div>

    <script>

        const obj = document.querySelector("#ouo");

        obj.innerHTML = "New content!!";

    </script>

</body>
```

修改 obj 的內部 HTML

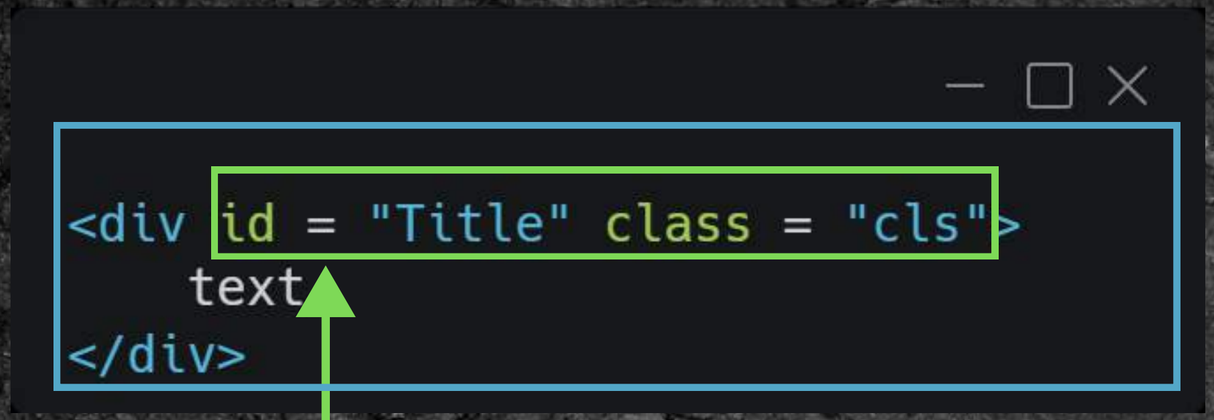


empty

New content!!



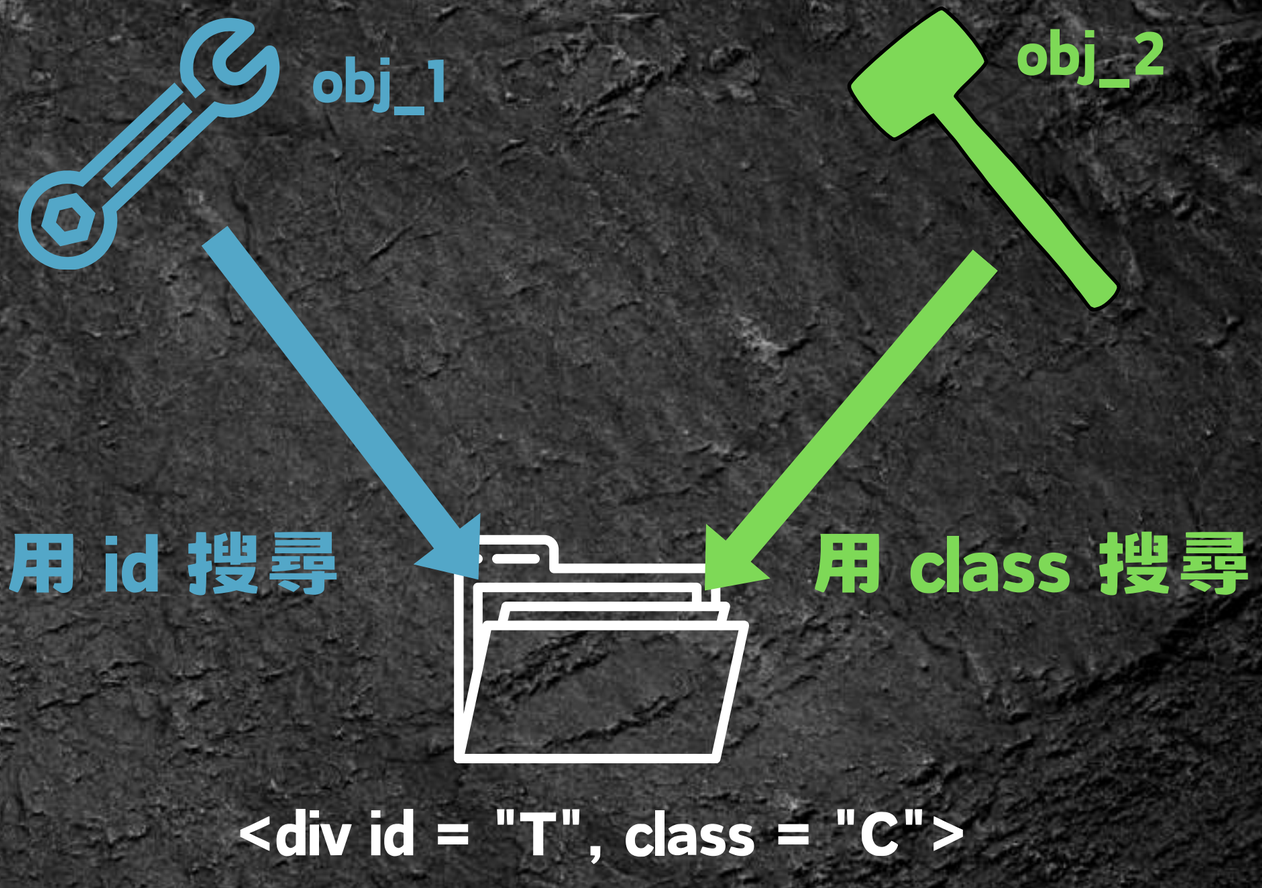
Element（元素）vs. Attribute（屬性）



- → HTML DOM 的 Element（元素）
- id 和 class → 的 Attribute（屬性）

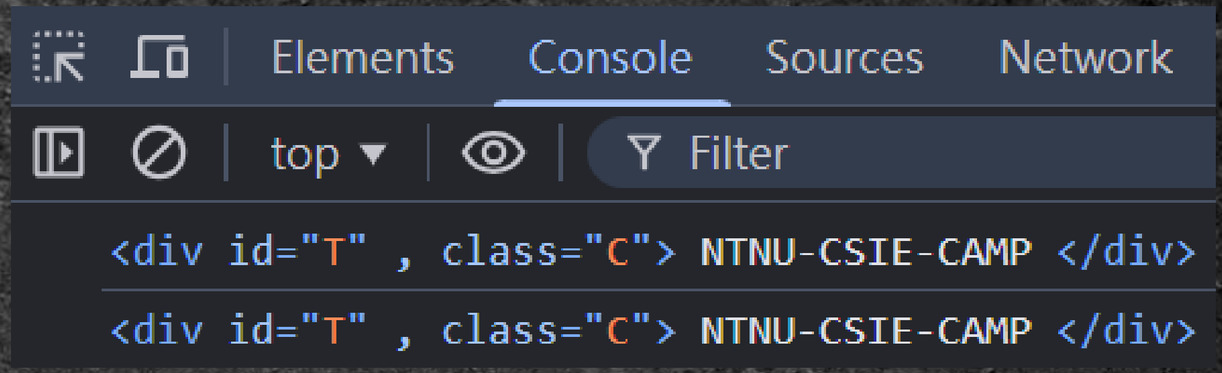
搜尋 HTML 物件: querySelector();

```
<body>

    <div id = "T", class = "C">
        NTNU-CSIE-CAMP
    </div>

    <script>

        const obj_1 = document.body.querySelector("#T");

        const obj_2 = document.body.querySelector(".C");

        console.log(obj_1);
        console.log(obj_2);

    </script>

</body>
```

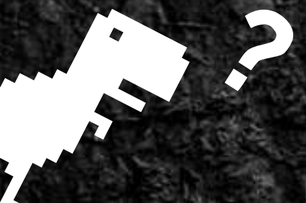


搜尋 HTML 物件: querySelector();

```
<body>

    <div id = "T", class = "C">
        NTNU-CSIE-CAMP
    </div>

    <script>

        const obj_1 = document.body.querySelector("#T");

        const obj_2 = document.body.querySelector(".C");

        console.log(obj_1);
        console.log(obj_2);

    </script>

</body>
```

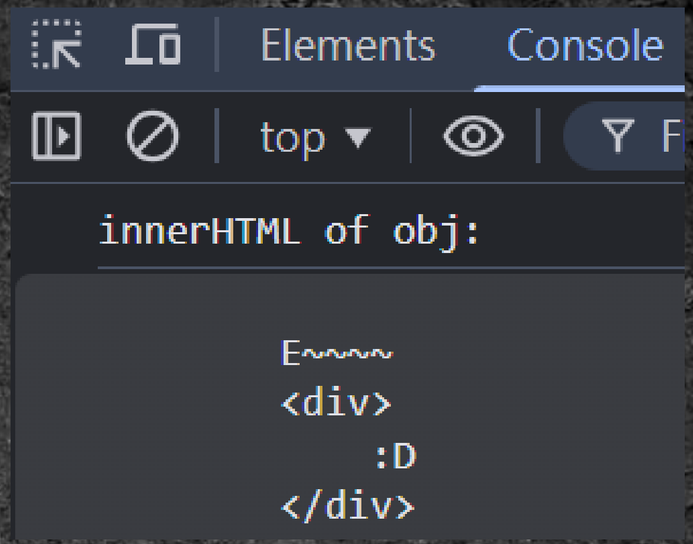


obj_1 和 obj_2 指向相同的物件



存取內部HTML：.innerHTML

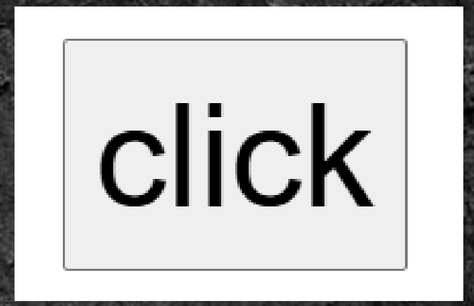
```
<body>

    <div id = "awa" >
        E~~~~
        <div>
            :D
        </div>
    </div>

    <script>
        const obj = document.querySelector("#awa");

        console.log("innerHTML of obj:");
        console.log( obj.innerHTML );

    </script>

</body>
```



- 取出 obj 指向的標籤內部的 HTML

按鈕 <button>

```
<button onclick = "myfunction()">click</button>
```



- onclick 事件: 按鈕被點擊 → 呼叫 myfunction()

- 事件: 使用者做的一些操作

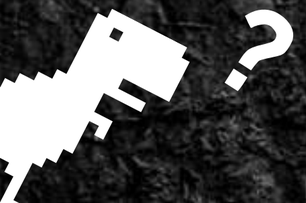
  e.g. 點擊滑鼠、 按鍵盤

按鈕 <button>

```
<body>

    <button id = "btn">click</button>

    <script>

        const obj = document.querySelector("#btn");
        let i  = 0;
        obj.onclick = function(){
            console.log(i++);
        };

    </script>
</body>
```

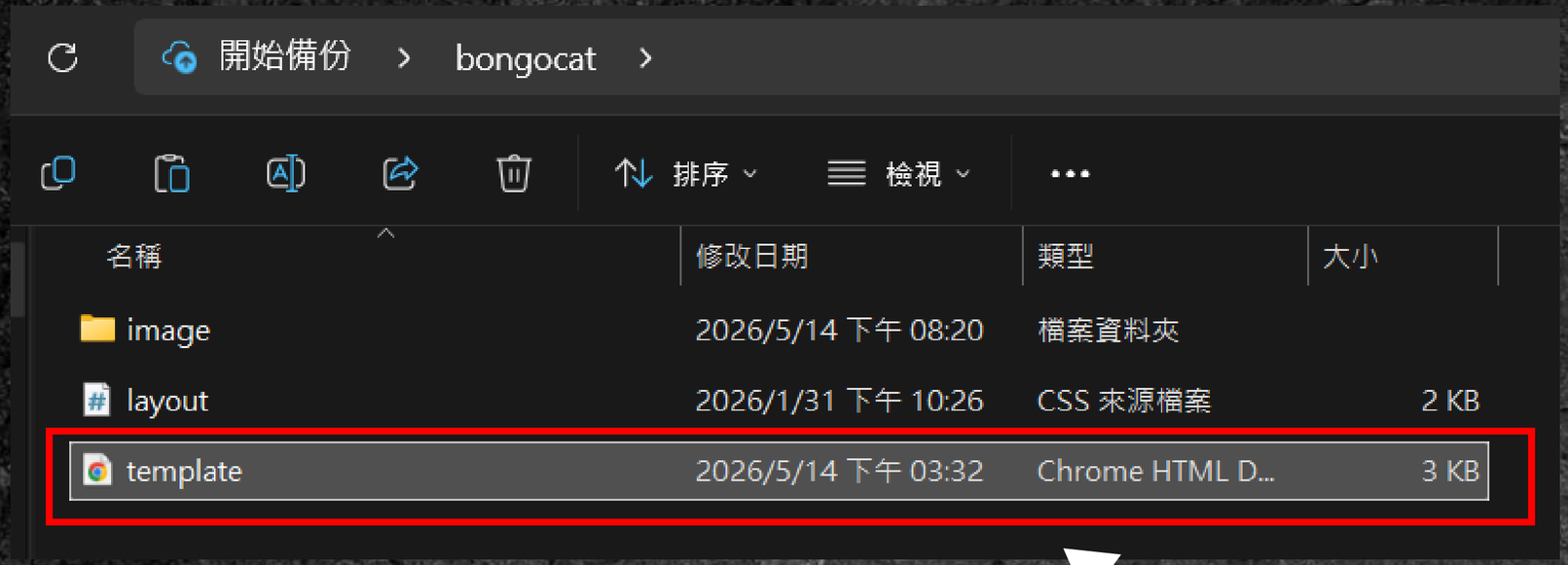
取出 obj 的 onclick 屬性

改變 onclick 屬性的函式 （功能）



Task 8

Bongocat~


開始備份 › bongocat ›
排序
檢視
名稱
修改日期
類型
大小
image
2026/5/14 下午 08:20
檔案資料夾
layout
2026/1/31 下午 10:26
CSS 來源檔案
2 KB
template
2026/5/14 下午 03:32
Chrome HTML D...
3 KB

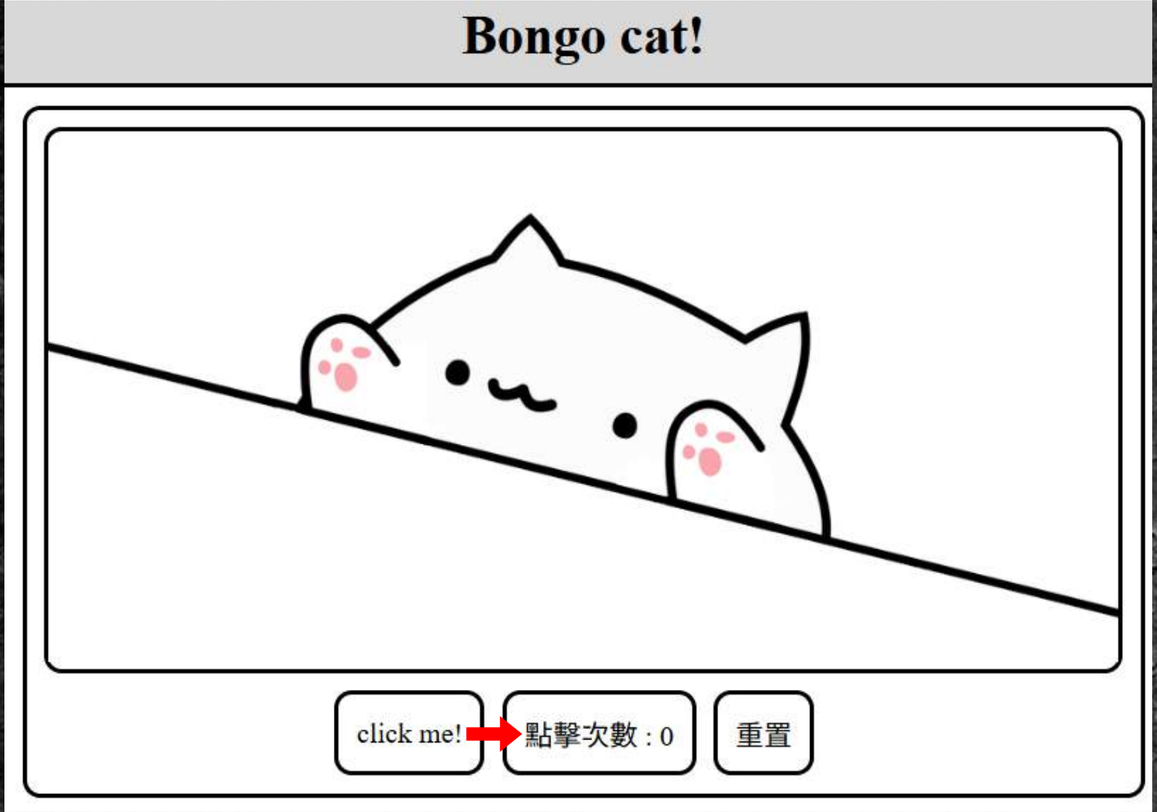
點開 Task 8 的模板


```
function click_function(){

    ??? //改變點擊次數的紀錄(cnt)
    ??? = "點擊次數 : " + String(cnt); //改變 <div class="count"> 的內容，String():強制把放

    change(); //改變顯示的圖片 -> 呼叫change()
}

/*
    把紀錄重置:
    1. 把點擊次數(cnt)歸零
    2. 把顯示的圖片重設為 "image/1.png"
*/
function reset(){
    ??? //把點擊次數(cnt)的紀錄歸零
    ??? = "點擊次數 : 0"; //改變 <div class="count"> 的內容
    ??? = "image/1.png"; //改變 <img id="image"> 的圖片
}

//用來改變圖片
function change() {

    idx = (idx+1) % Display_order.length;
    /*
        1. 把 idx 值加 1
        2. 如果原本的圖片是最後一個，則把 idx 重設為 0 (第一張圖片)
    */

    ??? = "image/" + String(Display_order[idx]) + ".png"; //改變 <img id="image"> 的圖片
```

???

我們要自己寫 JavaScript 的地方

Part 1（6分鐘）：改變點擊次數

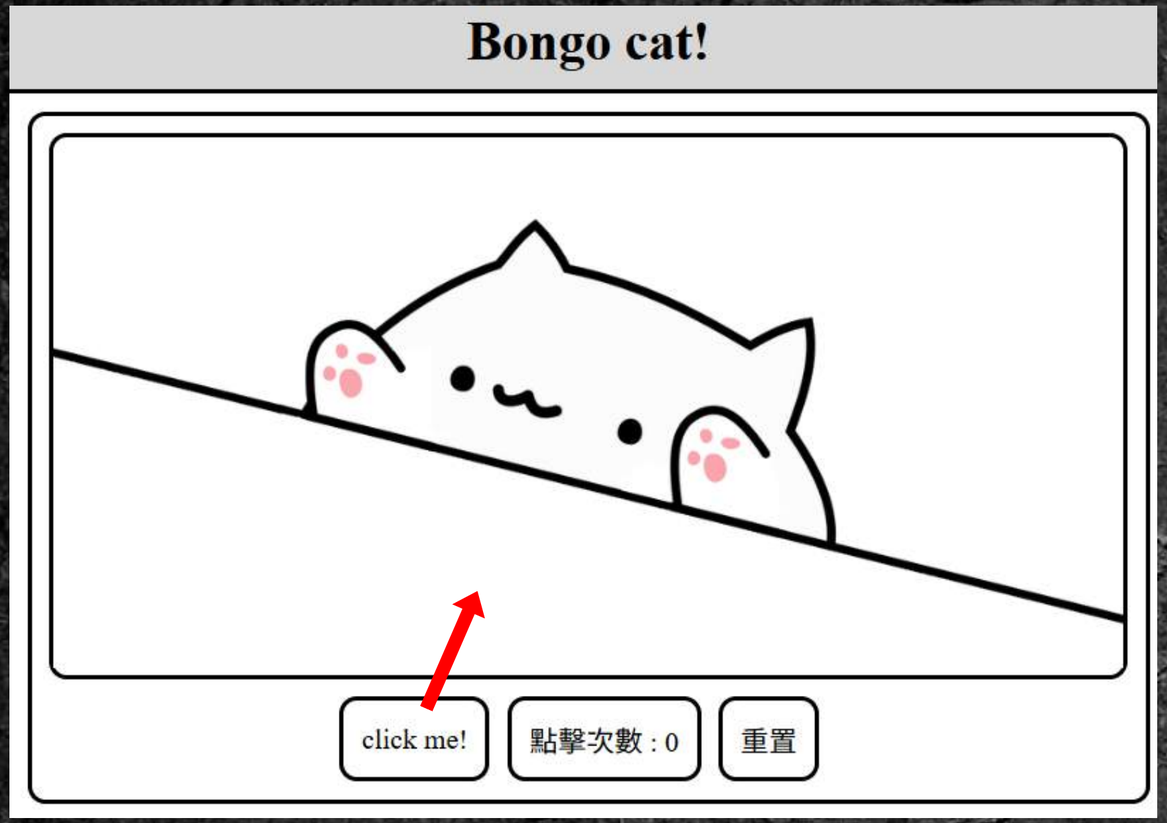


點擊按鈕 click me!

- 點按鈕 → 點擊次數 +1

```
onclick="f()" //按鈕被點擊(onclick) -> 呼叫 f()

let a = 0; // 一個數字變數 a
a += 1; //a的值增加 1

document.querySelector() //尋找 HTML 的標籤物件

obj.innerHTML //obj 的內部 HTML
```

Part 2（6分鐘）：改變圖片

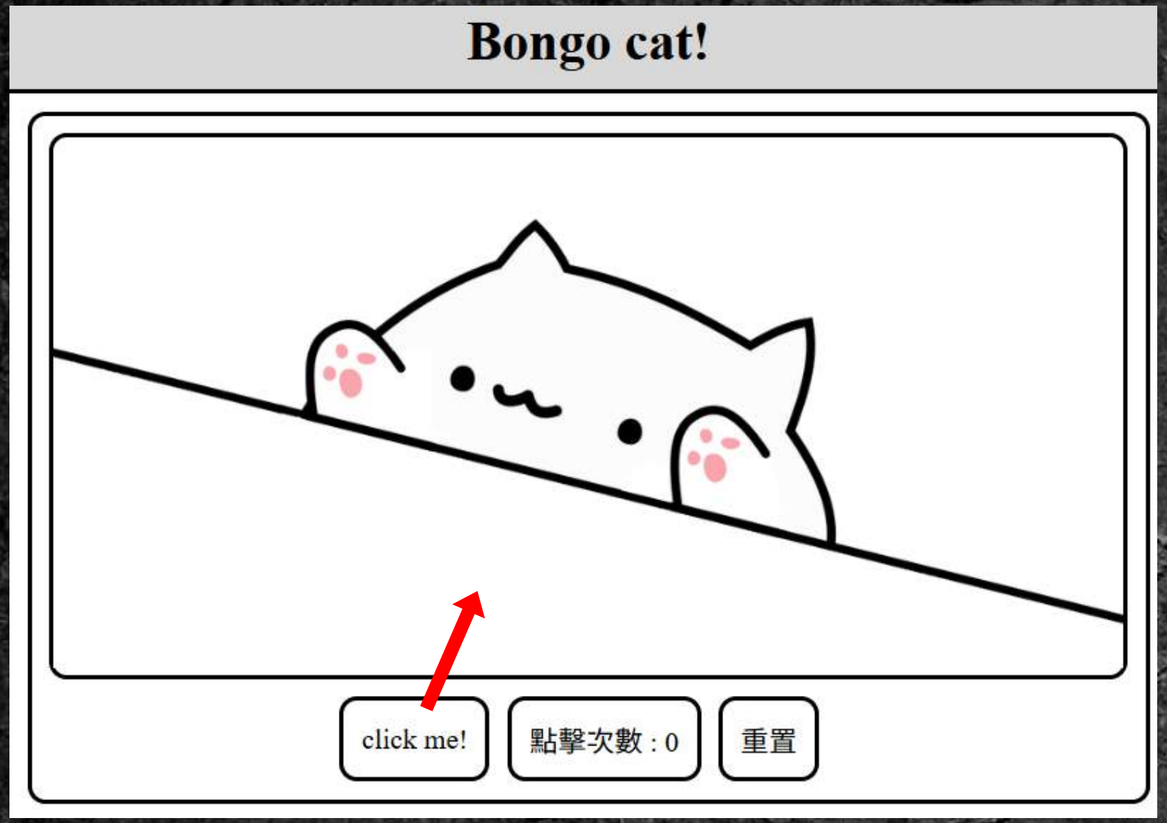


點擊按鈕 click me!

- 點按鈕 → 改變網頁的圖片

```
document.querySelector() //尋找 HTML 的標籤物件

obj.src //檔案路徑 -> 用來放圖片
```

Part 3（6分鐘）：重置按鈕



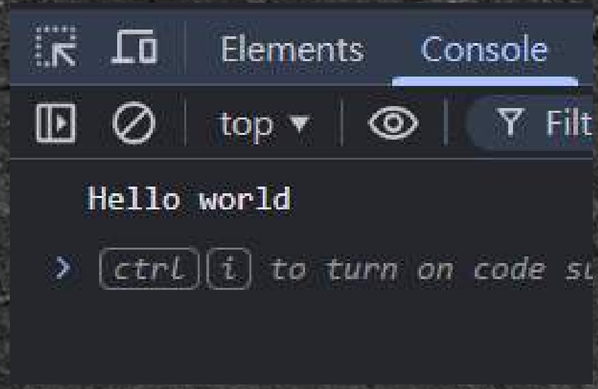
點擊按鈕「重置」

- 點按鈕：
  1. 點擊次數變回 0
  2. 變回一開始的圖片

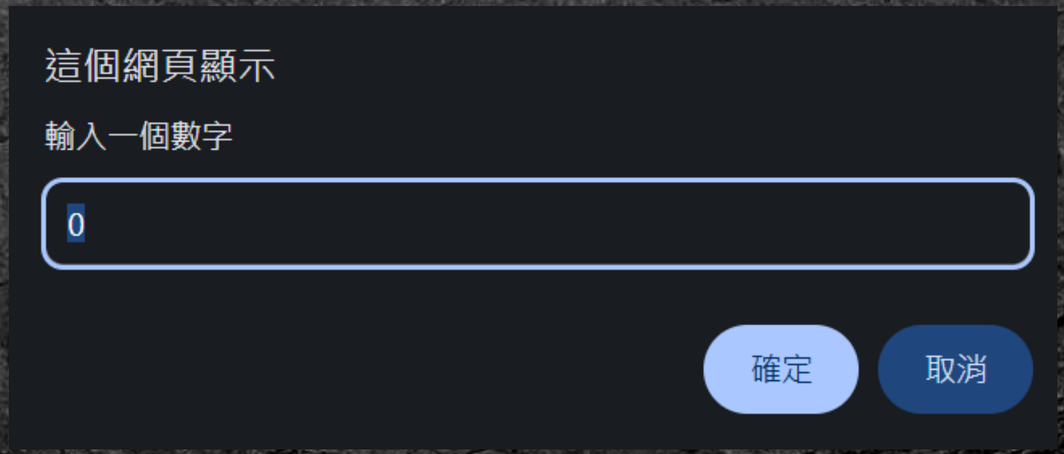
```
onclick="f()" //按鈕被點擊(onclick) -> 呼叫 f()

document.querySelector() //尋找 HTML 的標籤物件

obj.src //檔案路徑 -> 用來放圖片
```

總結

- 輸出 console.log()



- 輸入 prompt()

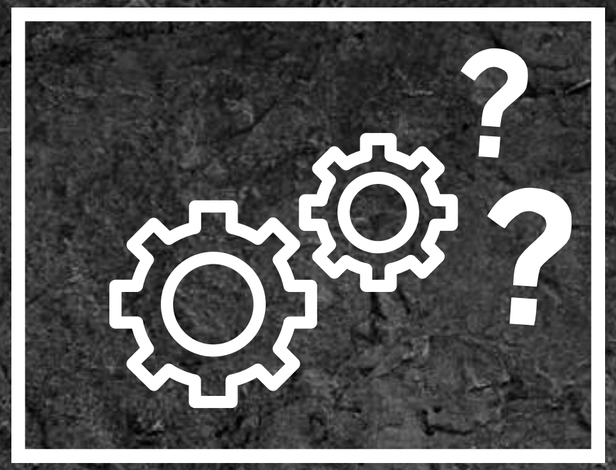


變數（Variables）

- 儲存資料的容器

字串（String）	"Hello world", 'ouob'
數字（Number）	1, 2147483647
布林值（Boolean）	true（對）, false（錯）

條件判斷式



true （正確）

執行程式碼

false （錯誤）

直接離開

- if、 else if 、 else

```
let a = Number(prompt());
if(a > 10){
    console.log("a > 10");
}
else if(a === 10){
    console.log("a = 10");
}
else{
    console.log("a < 10");
}
```

迴圈（loop）

- 用來執行程式碼中重複的操作
- 迴圈控制：break; continue;
- 小心無限迴圈!

迴圈 （loop）

for 迴圈

```
let i
for(i = 0 ; i < 5 ; i++){
    console.log(i);
}
console.log("current i =",i);
```

while 迴圈

```
let i = 0;
while(i < 10){
    i += 1;
    console.log("i =",i);
}
```

陣列（Array）

- 儲存同類型的資料
- 索引值（Index）
- 陣列搜尋：indexOf()、includes()

```
let arr = [123, 4, 98];

console.log("arr[2] =", arr[2]);

arr[2] = 100;

console.log("arr[2] =", arr[2]);
```

函式（Function）

- 常用的特殊功能 → 打包成函式
- 可以重複使用

參數 →  → 回傳的資料

（數字、文字、...）

物件（Object）

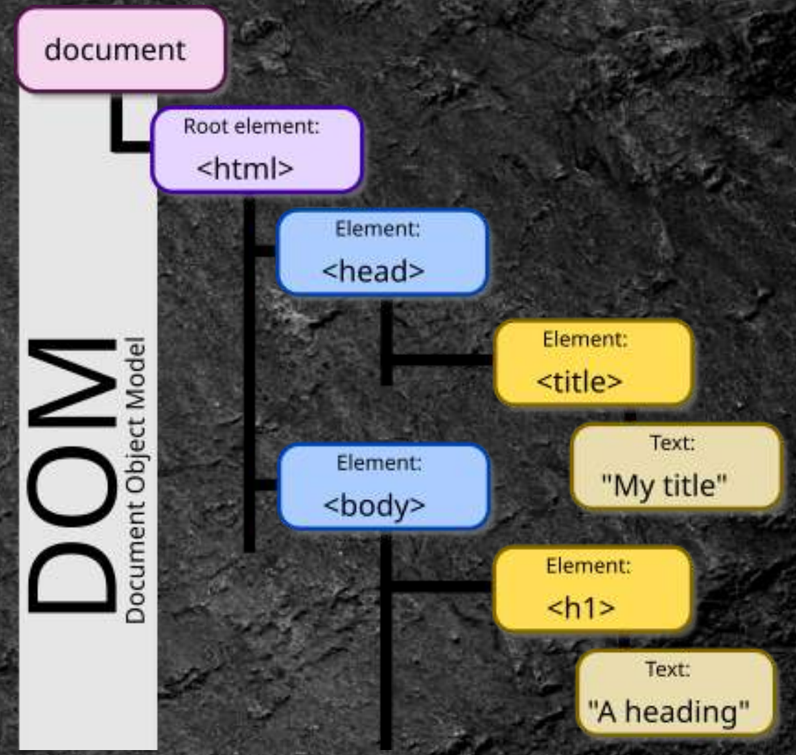
- 用來整合多個變數、陣列、函式

```
const Human = {
    name: "Ostrich",
    major: "CSIE",
    Height: 180,

    Sleep: function(){
        console.log("ZZZ...");
    },
    Eat: function(){
        //eat something
    }
}
```

- 名字：鴕鳥
- 大學的科系：資工系
- 身高：180
- 睡覺：Sleep();
- 打球：Play_ball();

HTML DOM

- HTML 標籤 → JS 的物件
- 取得物件: querySelector();
- 存取內部HTML: .innerHTML
- button 標籤



thanks for listening